

FAST 主动反射面形状调节的约束优化与接收性能分析

摘要

FAST 通过促动器带动索网节点,使局部球面在观测方向上形成近似旋转抛物面。其调节过程同时受促动器行程、下拉索定长和主索长度变化约束,理想面几何与工程可实现性不能分开处理。本文建立“理想面选择—严格可行调节—有限面板接收评价”的统一模型。

对问题一,固定馈源焦点,在一参数旋转抛物面族中比较几何焦距、面积加权最小二乘焦距和硬件友好焦距,以促动器与主索最大归一化占用率为选择准则。正上方观测时得到硬件友好焦距 140.325121 m;理想节点直接投影的主索占用率为上限的 1.5815 倍,说明必须进一步求可行工作面。

对问题二,以 692 个工作区节点的径向位移为变量,建立面积加权平方贴合目标和 2165 条相关主索、692 个促动器的精确约束。采用三个严格可行起点的顺序凸化方法,并在每个候选步后回代原始非线性几何;随后在均方误差容差内精修最大节点误差。指定观测方向下,理想抛物面顶点为 $(-49.375757, -36.931066, -294.350722)$ m。最终节点贴合理想面的均方根误差为 0.062858 m,最大主索相对变化为 0.069999226%,最大促动器绝对伸缩量为 0.249123 m,全部硬约束均满足。

对问题三,将每块三角面板作确定性等面积细分,以三顶点平面法向计算镜面反射,并按入射投影面积加权。64 阶细分下,基准球面和调节工作面的接收比分别为 0.00811012 和 0.01075298,工作面相对提高 32.5872%。从 32 阶增加到 64 阶时,工作面接收比仅变化 1.40×10^{-5} ;改变投影口径半径和面板权重后,提升方向保持一致。结果表明,严格可行调节虽受主索约束主导,仍能稳定改善有限馈源条件下的接收性能。

关键字: FAST 主动反射面 顺序凸化 几何光学 射线追踪

一、问题重述

1.1 问题背景

五百米口径球面射电望远镜 FAST 的主反射面由三角形反射面板、柔性索网、下拉索和促动器组成。基准状态近似为半径约 300 m、口径 500 m 的球面。观测特定方向的天体时，促动器改变局部索网节点位置，使 300 m 有效口径内的球面变形为近似旋转抛物面，以减小球差并把平行入射电磁波汇聚到馈源舱。FAST 的工程构型和主动主反射面原理可参见南仁东等的项目论文。²³

调节不能只追求几何上的理想抛物面。每个促动器的伸缩量限制在 $[-0.6, 0.6]$ m，任意相邻主索节点距离的相对变化不超过 0.07%，同时每根下拉索保持原长。反射面还由有限尺寸的三角面板组成，馈源有效区域只是直径 1 m 的圆盘，因此接收效果需要在实际离散几何上评价。

1.2 问题内容

问题一要求在天体位于基准球面正上方时，结合焦面位置和调节条件，确定理想旋转抛物面。需要给出能够唯一描述该抛物面的参数或顶点，而不能只写出一族满足焦点条件的曲面。

问题二给定天体方位角 $\alpha = 36.795^\circ$ 、仰角 $\beta = 78.169^\circ$ ，要求确定理想抛物面，并计算 300 m 工作口径内主索节点的调整后坐标和相应促动器伸缩量。结果必须逐条满足促动器、下拉索和主索约束，并按规定表格输出。

问题三要求在相同观测方向和 300 m 投影口径下，比较问题二调节工作面与基准球面对馈源有效圆盘的接收比。比较过程应采用一致的面板离散、入射功率权重和命中判断。

三个问题依次传递信息：问题一确定理想面的选择原则，问题二把理想目标投影到工程可行域，问题三以问题二的正式几何作为唯一工作面输入。

二、问题分析

2.1 理想面与可行工作面必须分层

题目给出的焦径比首先确定基准球面与焦面之间的距离，但固定焦点仍允许改变抛物面焦距和顶点位置。若直接令抛物面顶点位于基准球面，模型虽然解析简单，却没有利用促动器和主索的实际调节裕度。另一方面，只按平均位移最小选择抛物面，也可能让少数边率先触碰硬约束。因此问题一应在固定焦点的一参数抛物面族中比较多种定义，并以最大硬件占用率作为主要选择依据。

即使理想面的焦距已经兼顾硬件，其节点投影也未必同时满足全部离散主索约束。本题实际计算显示，理想目标的促动器行程有余量，但最大主索变化超过允许值。这一现象说明“确定理想曲面”和“求可实现工作面”是两个不同层次：前者给出物理目标，后者是在精确约束下求离目标最近的节点配置。

2.2 稀疏约束优化结构

问题二包含约 700 个连续节点位移变量和约 2000 条局部边约束。每条边只涉及两个端点，促动器约束只涉及单个节点，因此约束雅可比具有明显稀疏性。罚函数会把“约束违反量较小”与“真正可行”混在一起，直接大规模非线性规划又容易依赖初值。本文选择从原始球面严格可行点出发，对精确几何作局部线性化，求解稀疏凸二次子问题，再对

候选步进行原始非线性回代。凸子问题的建模和求解依据凸优化理论及 CVXPY、OSQP 的公开方法。⁴⁵⁶

2.3 有限面板接收评价

若只追踪节点或面板中心的一条射线，会忽略部分面板位于口径外、部分面板只有局部能量命中馈源的情况。本文把三角面板作确定性等面积细分，在每个微单元重心发射反射线，并用入射投影面积作为能量权重。细分级别逐次加倍，接收比的变化直接提供离散误差证据。

下图概括了数据、三个问题及验证环节之间的关系。

FAST 主动反射面建模技术路线

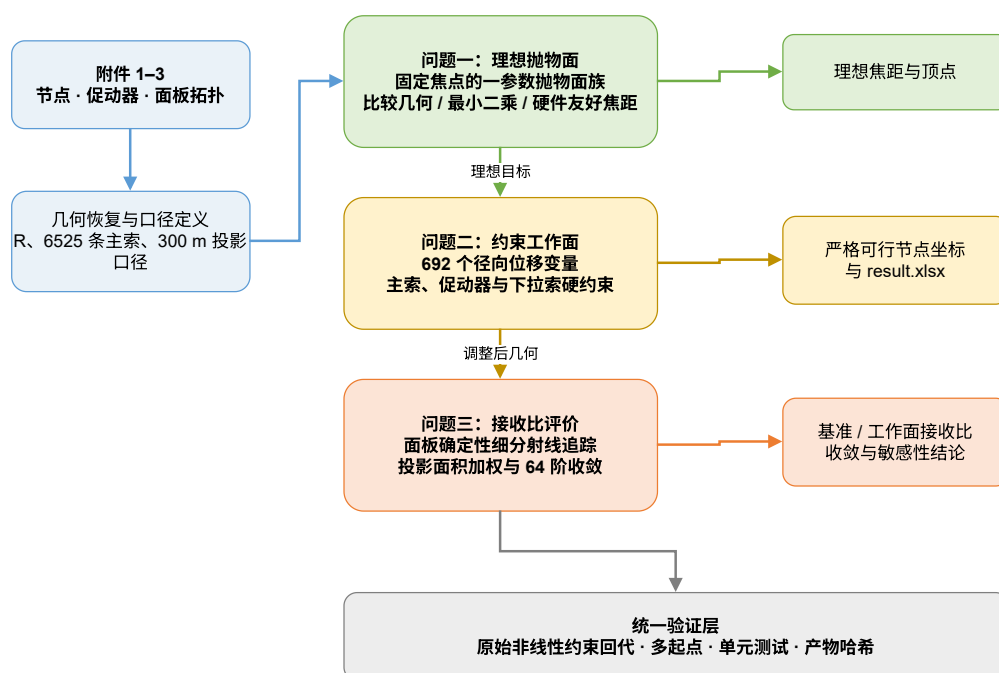


图 1 FAST 主动反射面三问统一建模技术路线

由图 1 可见，焦距选择、工作面优化和接收比评价共享同一套坐标、口径和几何约定；每一层的输出都经过原始约束或收敛性复核后才进入下一层。

三、模型假设

1. 基准球心位于坐标原点。球面半径使用附件节点半径的均值，而不把题面中的“约 300 m”强制取整。这样能够保留附件坐标的内部一致性。
2. 节点只沿基准球心径向运动。促动器按基准球面径向安装，而附件没有给出索网预张力、材料刚度和节点三维自由度，因此径向位移是当前数据下可辨识且与装置结构一致的控制变量。该假设忽略了受力引起的微小切向偏移。
3. 下拉索逐根保持基准长度。不同节点的下拉索长度由附件几何分别恢复，不能用统一长度替代；促动器顶端沿自身基准径向移动。
4. 口径外节点固定。300 m 口径由基准位置对观测轴的投影圆选取。口径外节点位移为零，但所有连接活动节点的跨边界主索仍满足长度变化约束。
5. 反射面板是三顶点决定的局部平面镜。面板间缝隙避免相邻面板接触造成额外变形。该假设未描述面板自身柔性、表面粗糙度和装配误差。

6. 电磁波作平行直线传播。面板无吸收，反射服从几何光学镜面定律；暂不考虑衍射、遮挡、馈源方向图和极化。基准面与工作面使用完全一致的能量口径，因此这些忽略主要影响接收比绝对值，不改变本文的同口径比较定义。
7. 附件坐标和拓扑正确。对缺失字段、重复编号、未知面板节点和零面积面板均在程序入口拒绝。坐标舍入与口径选择的影响通过灵敏度实验讨论。
8. 优化结果不宣称全局最优。顺序凸化属于局部方法。本文采用三个确定性可行起点并逐条回代硬约束，将结果称为“多起点中的最佳严格可行解”。

四、符号与坐标系

主要符号见 表 1。

表 1 主要符号说明

符号	含义	单位
R_i	节点 i 的基准半径	m
R	全部节点基准半径的均值	m
F	题面给定的焦面距离参数, $F = 0.466R$	m
q	焦面球半径, $q = R - F$	m
$\mathbf{u}$	从球心指向天体的单位向量	—
$\mathbf{k}$	工作反射面轴向单位向量, $\mathbf{k} = -\mathbf{u}$	—
$\mathbf{P}$	馈源平面中心和抛物面焦点	m
$\mathbf{n}_i$	节点 i 的基准径向单位向量	—
y_i	节点沿 $\mathbf{n}_i$ 向球心的位移	m
σ_i	促动器顶端向球心的伸缩量	m
L_i	第 i 根下拉索基准长度	m
l_{ij}	主索边 (i, j) 的基准长度	m
ε	主索相对变化上限, 0.0007	—
w_i	由相邻面板面积分配得到的节点权重	m ²

方位角 α 和仰角 β 转换为

$$\mathbf{u} = (\cos \beta \cos \alpha, \cos \beta \sin \alpha, \sin \beta), \quad \mathbf{k} = -\mathbf{u}. \quad (1)$$

焦面中心取为

$$\mathbf{P} = q\mathbf{k}, \quad q = R - F. \quad (2)$$

对任意坐标 $\mathbf{x}$, 定义沿工作面轴的坐标与离轴距离

$$t = \mathbf{x} \cdot \mathbf{k}, \quad \rho(\mathbf{x}) = \sqrt{\|\mathbf{x}\|^2 - t^2}. \quad (3)$$

300 m 工作节点集合固定按基准位置选择:

$$\mathcal{I}(\mathbf{k}) = \{i : \mathbf{x}_i \cdot \mathbf{k} > 0, \rho(\mathbf{x}_i) \leq 150\}. \quad (4)$$

节点调整后坐标为

$$\mathbf{x}_{i(y_i)} = \mathbf{x}_i - y_i \mathbf{n}_i. \quad (5)$$

全文约定 $y_i > 0$ 和 $\sigma_i > 0$ 都表示向球心方向移动。口径外节点令 $y_i = 0$, 但与活动节点关联的主索仍进入约束集合。

五、 问题一：理想抛物面的确定

5.1 固定焦点的一参数抛物面

数据恢复的平均基准球半径为 $R = 300.400011$ m，题面焦面距离参数为 $F = 0.466R$ 。正上方观测时取 $\alpha = 0^\circ, \beta = 90^\circ$ ，工作面轴 $\mathbf{k}$ 指向反射面一侧，焦点固定为 $\mathbf{P} = (R - F)\mathbf{k}$ 。

令旋转抛物面的焦距为 $f > 0$ ，顶点沿轴向的半径为

$$v = q + f, \quad q = R - F, \quad \mathbf{V} = v\mathbf{k}. \quad (6)$$

在以 $\mathbf{k}$ 为轴的局部坐标中，候选抛物面写为

$$\rho(\mathbf{x})^2 = 4f[v - \mathbf{x} \cdot \mathbf{k}]. \quad (7)$$

当 $f = F$ 时有 $v = R$ ，顶点位于基准球面。固定焦点后只剩焦距 f 一个自由度，避免了同时任意移动焦点和顶点造成的不唯一性。

5.2 节点目标位移

节点 i 的径向射线记为 $\mathbf{x} = \lambda\mathbf{n}_i$ 。令

$$c_i = \mathbf{n}_i \cdot \mathbf{k}, \quad a_i = 1 - c_i^2, \quad (8)$$

把径向射线代入式 7，得到

$$a_i \lambda_i^2 + 4f c_i \lambda_i - 4fv = 0. \quad (9)$$

从正根中选取最接近基准半径 R_i 的交点。轴线上 $a_i = 0$ 时使用线性极限 $\lambda_i = \frac{v}{c_i}$ 。理想目标位移为

$$\mathbf{y}_i^{*(f)} = R_i - \lambda_{i(f)}, \quad i \in \mathcal{I}. \quad (10)$$

节点面积权重 w_i 由每块相邻三角面板面积各分配三分之一得到。这样，内圈连接面板较多的节点不会仅因拓扑度数而被重复计权。

5.3 下拉索与促动器映射

记促动器下端点为 $\mathbf{a}_i$ ，基准态上端点为 $\mathbf{b}_i$ ，促动器向外径向单位向量为 $\mathbf{h}_i$ 。基准下拉索长度为

$$L_i = \|\mathbf{x}_i - \mathbf{b}_i\|. \quad (11)$$

促动器顶端向球心伸缩 σ_i 后的位置是 $\mathbf{b}_i - \sigma_i \mathbf{h}_i$ 。下拉索定长条件为

$$\|\mathbf{x}_i - y_i \mathbf{n}_i - \mathbf{b}_i + \sigma_i \mathbf{h}_i\|^2 = L_i^2. \quad (12)$$

令 $\mathbf{d}_{i(y)} = \mathbf{x}_i - \mathbf{b}_i - y \mathbf{n}_i$ ，则式 12 等价于

$$\sigma_i^2 + 2[\mathbf{h}_i \cdot \mathbf{d}_{i(y)}] \sigma_i + \|\mathbf{d}_{i(y)}\|^2 - L_i^2 = 0. \quad (13)$$

二次方程可能有两个根。本文选取在 $y = 0$ 时等于零、随 y 连续的根；判别式为负时，该目标节点在当前运动假设下不可达。

5.4 硬件友好焦距

对每个候选焦距，把 $y_i^{*(f)}$ 代入促动器映射和所有关联主索，定义

$$U_{\text{act}(f)} = \max_{i \in \mathcal{I}} \frac{|\sigma_i(y_i^*)|}{0.6}, \quad (14)$$

$$U_{\text{edge}(f)} = \max_{(i,j) \in \mathcal{E}_T} \frac{\left| \frac{\|\mathbf{x}_{i(y_i^*)} - \mathbf{x}_{j(y_j^*)}\|}{l_{ij}} - 1 \right|}{0.0007}, \quad (15)$$

$$U(f) = \max(U_{\text{act}(f)}, U_{\text{edge}(f)}). \quad (16)$$

先在 $f = F$ 附近作确定性网格搜索，再对最优邻域作一维有界精化。若多个焦距的 $U(f)$ 在数值容差内相同，则以面积加权位移均方误差作为第二判据。候选结果见 表 2。
表 2 正上方观测的理想抛物面候选比较

候选	f/m	位移 MSE/m^2	U_{act}	U_{edge}
几何焦距	139.986405	0.262893	1.159033	3.302666
最小二乘	140.386402	0.041546	0.666681	1.873045
硬件友好	140.325121	0.051657	0.564544	1.581528

图 2 给出两个观测方向下焦距与最大硬件占用率的关系。曲线最小点仍高于 1，且主索占用率明显大于促动器占用率，表明改变焦距只能缓解而不能消除离散索网冲突。

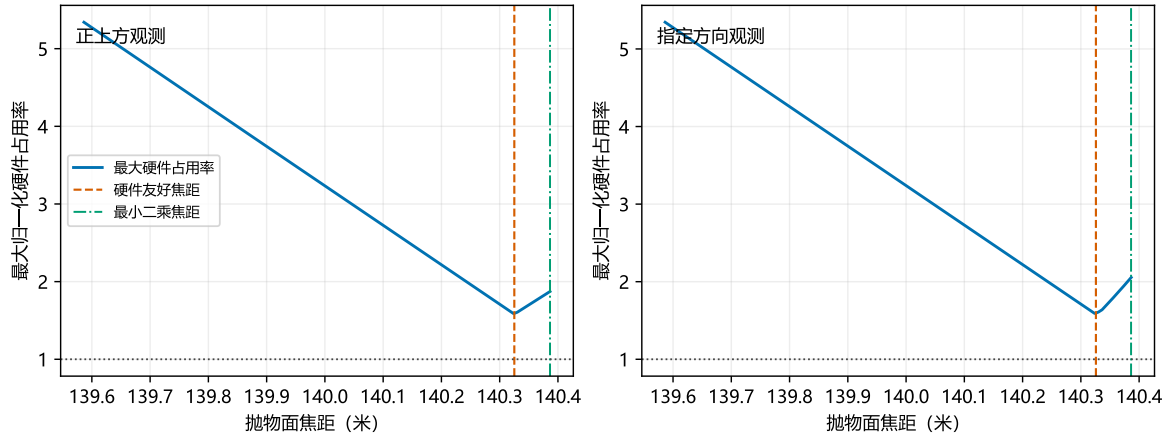


图 2 两个观测方向的焦距与最大硬件占用率

综上，问题一取硬件友好焦距

$$f_1^* = 140.325121 \text{ m}, \quad (17)$$

对应顶点轴向半径 $v_1 = 300.738727 \text{ m}$ ，顶点坐标为

$$\mathbf{V}_1 = (-0.000000, -0.000000, -300.738727) \text{ m}. \quad (18)$$

该结果定义了工程意义下的理想目标，但 表 2 中 $U_{\text{edge}} > 1$ 说明目标节点不能直接作为实际工作面。

六、问题二：严格可行工作面优化

6.1 指定方向的理想目标

对 $\alpha = 36.795^\circ, \beta = 78.169^\circ$ 重新构造 $\mathbf{u}, \mathbf{k}, \mathbf{P}$ 和投影口径。离散节点分布相对观测轴并非严格旋转对称，因此不能简单旋转问题一的数值结果，而应使用同一硬件友好准则重新搜索焦距。

指定方向下得到

$$f_2^* = 140.325843 \text{ m}, \quad (19)$$

$$\mathbf{V}_2 = (-49.375757, -36.931066, -294.350722) \text{ m}. \quad (20)$$

投影口径内有 692 个活动节点，与它们关联的主索为 2165 条，其中包含跨口径边界的主索。理想目标本身的最大促动器伸缩量为 0.338250 m，但最大主索相对变化为 0.110593%，超过 0.07% 上限，因此进入约束工作面优化。

6.2 优化模型

决策变量为活动节点的径向位移 $\mathbf{y} = (y_i)_{i \in \mathcal{I}}$ 。口径外节点固定为零。第一阶段最小化面积加权贴合误差

$$\min_{\mathbf{y}} J(\mathbf{y}) = \frac{\sum_{i \in \mathcal{I}} w_i^2 (y_i - y_i^*)}{\sum_{i \in \mathcal{I}} w_i}. \quad (21)$$

对每个活动节点，促动器连续根必须存在且满足

$$-0.6 \leq \sigma_{i(y_i)} \leq 0.6. \quad (22)$$

对所有与活动节点关联的主索，要求

$$(1 - \varepsilon)l_{ij} \leq \|\mathbf{x}_i - y_i \mathbf{n}_i - \mathbf{x}_j + y_j \mathbf{n}_j\| \leq (1 + \varepsilon)l_{ij}, \quad (23)$$

其中跨边界端点的 $y_j = 0$ 。约束式 22 和式 23 均在调整后坐标上精确计算，而不是只检查线性化子问题。

得到最优加权误差 J^* 后，在

$$J(\mathbf{y}) \leq J^{*(1+10^{-6})} + 10^{-12} \quad (24)$$

条件下求

$$\min_{\mathbf{y}, z} z, \quad |y_i - y_i^*| \leq z, \quad (25)$$

从而在几乎不损失平均贴合度的前提下降低最坏节点误差。

6.3 多起点顺序凸化

直接目标位移不可行，而零位移基准球面严格可行。首先沿目标射线二分求最大可行缩放比例，再构造零位移、半缩放目标和最大可行缩放目标三个起点。对每个起点重复以下过程：

1. 在当前严格可行点处，对主索长度和促动器映射作一阶线性化；
2. 在信赖域内求解稀疏凸二次子问题；
3. 把候选位移代回下拉索、促动器和主索的原始非线性几何；
4. 若候选严格可行且目标下降，则接受步长；否则回溯并缩小信赖域；
5. 当最大位移步长或目标改进低于阈值时停止。

该流程如图 3 所示。凸子问题由 CVXPY 建模，L2 阶段使用 OSQP 求解； L^∞ 精修使用锥优化后端。求解器状态只用于诊断，最终可行性以原始几何回代为准。

约束工作面顺序凸化求解流程

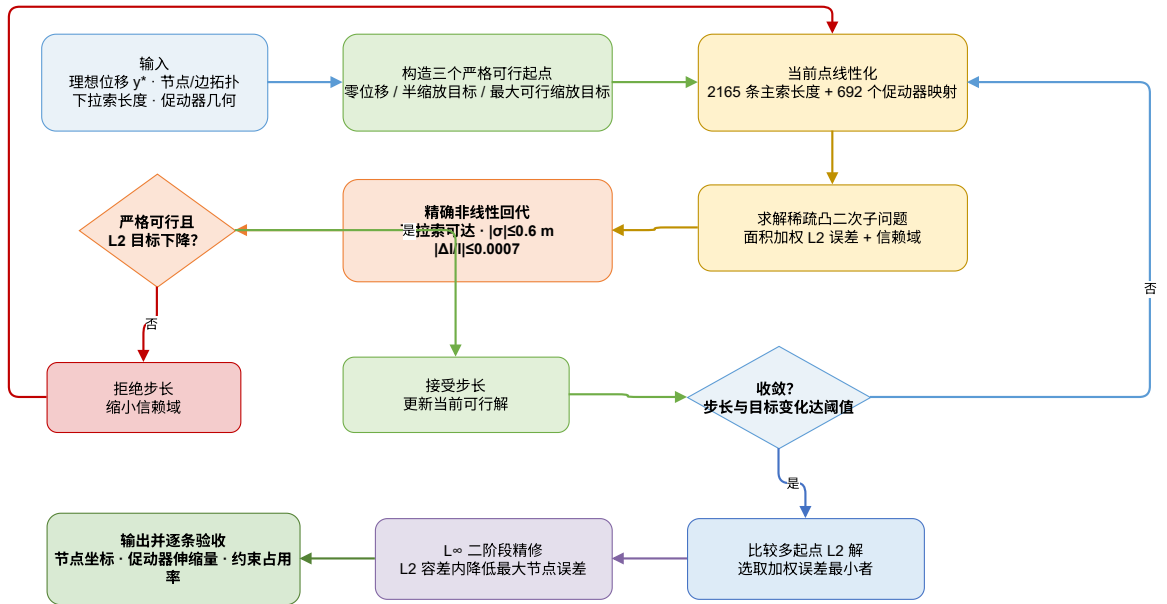


图 3 严格可行顺序凸化与二阶段精修流程

6.4 调节结果

三个起点均得到可行迭代序列，其中零位移起点的 L2 目标最小。正式结果见表 3。

表 3 指定方向下严格可行工作面的主要结果

指标	数值	指标	数值
面积加权 MSE/m ²	0.00395116	RMSE/m	0.062858
最大节点误差/m	0.132109	活动节点数	692
位移最小值/m	-0.249123	位移最大值/m	0.210278
伸缩最小值/m	-0.249123	伸缩最大值/m	0.210278
促动器占用率	0.415206	主索占用率	0.999988950

图 4 左图把理想目标和约束优化后的实际位移按离轴距离排列。中部节点基本贴合理想面，而靠近口径边缘的节点需要偏离目标以保护跨边界主索。右图显示促动器伸缩量集中在约 -0.25 至 0.21 m，远未触碰 ± 0.6 m 行程上限。

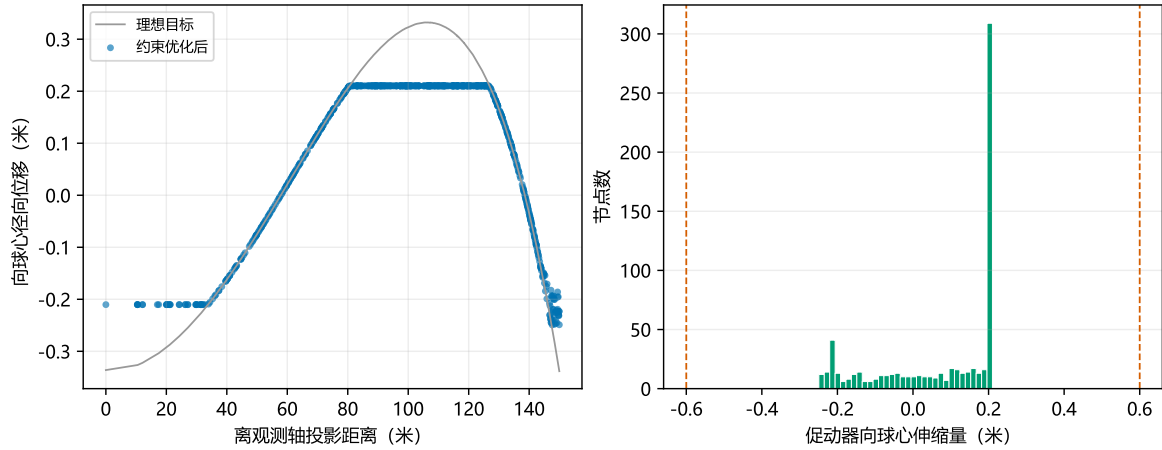


图 4 理想/实际节点位移与促动器伸缩量分布
全部 2165 条相关主索均逐条回代。按绝对相对变化排序的最紧五条边见 表 4。
表 4 最接近主索变化上限的五条边

左节点	右节点	相对变化/%	绝对变化/%
A94	A95	0.069999226	0.069999226
A93	A94	0.069999221	0.069999221
D265	D266	0.069999195	0.069999195
D163	D164	-0.069999194	0.069999194
D152	D170	-0.069999193	0.069999193

图 5 进一步给出全部边的约束占用分布。大量边落在上限附近，说明主索而非促动器是决定工作面形状的活跃约束。最大相对变化为 $0.069999226\% < 0.07\%$ ，仍位于允许范围内。

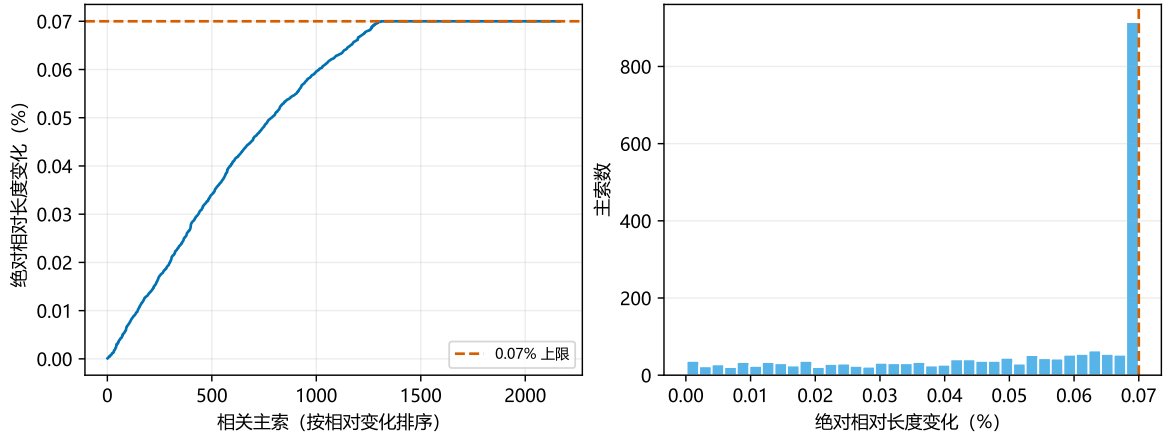


图 5 相关主索绝对相对长度变化及分布

最终把理想顶点、692 个调整后节点坐标和相同顺序的促动器伸缩量写入规定的三张结果表。内部计算保留双精度，表中显示六位小数；写出后重新读取首尾行并核对编号顺序。

七、问题三：馈源接收比

7.1 面板反射几何

对基准面或问题二的调节工作面，每块三角面板由三个节点坐标确定。设入射电磁波传播方向为 $\mathbf{d} = \mathbf{k}$ ，三顶点叉积得到单位法向 $\mathbf{m}$ 。面板顶点顺序不保证一致，因此把法向统一翻到天体侧，使 $\mathbf{d} \cdot \mathbf{m} < 0$ 。镜面反射方向为

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{m})\mathbf{m}. \quad (26)$$

馈源接收平面通过 $\mathbf{P}$ ，法向为 $\mathbf{u} = -\mathbf{k}$ 。从面板采样点 $\mathbf{p}$ 出发的反射线与馈源平面交于

$$\tau = \frac{(\mathbf{P} - \mathbf{p}) \cdot \mathbf{u}}{\mathbf{r} \cdot \mathbf{u}}, \quad \mathbf{q}_p = \mathbf{p} + \tau \mathbf{r}. \quad (27)$$

只有 $\tau > 0$ 且

$$\|\mathbf{q}_p - \mathbf{P}\| \leq 0.5 \text{ m} \quad (28)$$

时，该采样微单元被有效馈源圆盘接收。

7.2 确定性面板细分

仅用面板中心会使命中判定对三角形边界过于敏感。本文把每块母三角形作 m 阶等面积细分，共得到 m^2 个微三角形，并以各微三角形重心代表位置。所有微单元沿用母面板法向；跨越 300 m 口径边界的面板按微单元重心是否满足 $\rho(\mathbf{p}) \leq 150 \text{ m}$ 部分计入。

微单元 s 的能量权重取入射投影面积

$$\omega_s = A_s |\mathbf{d} \cdot \mathbf{m}_s|. \quad (29)$$

接收比定义为

$$\eta = \frac{\sum_s \omega_s \mathcal{I}_s}{\sum_s \omega_s}, \quad (30)$$

其中 $\mathcal{I}_s$ 是由式 27 给出的命中指示量。投影面积加权是正式口径，面板等权只用于敏感性对照。整体计算结构见图 6。

面板级接收比计算模型

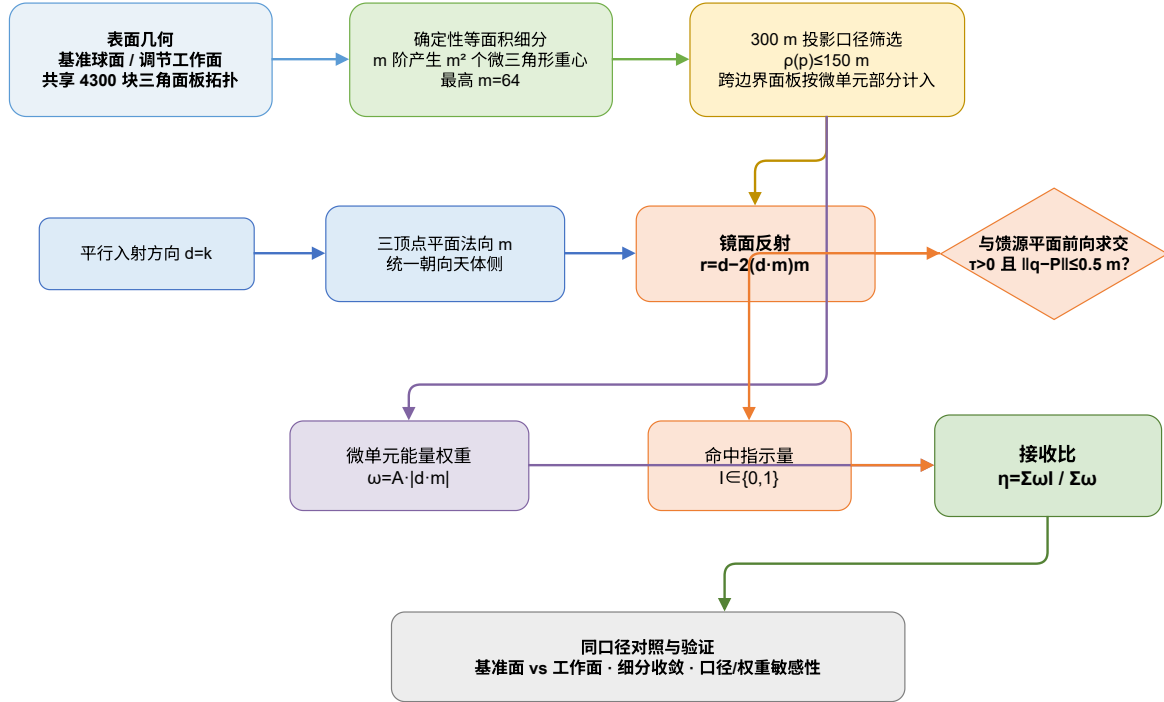


图 6 面板确定性细分与馈源接收比计算结构

7.3 细分收敛

对基准球面和调节工作面分别取 $m = 2, 4, 8, 16, 32, 64$, 结果见表 5。

表 5 三角面板细分阶数与接收比

细分阶数 m	基准面接收比	工作面接收比
2	0.01558945	0.02659522
4	0.01006153	0.01256800
8	0.00786994	0.00781574
16	0.00800792	0.01126826
32	0.00811106	0.01076696
64	0.00811012	0.01075298

粗细分时, 少量微单元跨过半径 0.5 m 的命中边界会导致比例明显跳动; 当 m 增加到 32 和 64 后, 两表面均趋于稳定。基准面从 32 阶到 64 阶的变化为 9.42×10^{-7} , 工作面变化为 1.40×10^{-5} , 分别约占最终值的 0.012% 和 0.130%。

图 7 左图显示收敛过程, 右图给出面板中心反射线在馈源平面的落点。调节面在中心附近形成更密集的落点, 但由于每块约 10 m 尺度的三角面板仍按一个平面法向反射, 而馈源半径仅 0.5 m, 所以在本文的有限平面面板模型下, 接收比绝对值仍较小。

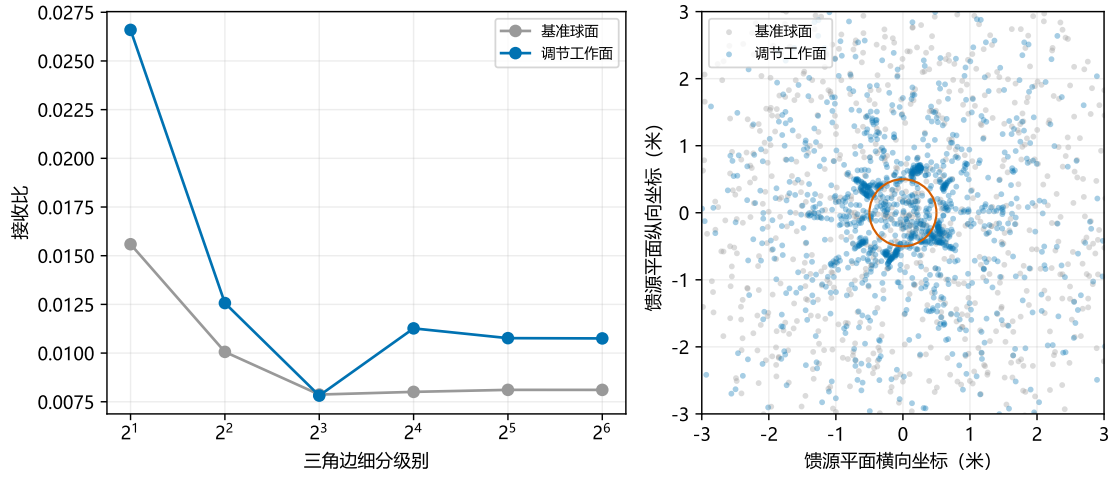


图 7 细分收敛与馈源平面反射落点

7.4 接收比比较

64 阶投影面积加权下,

$$\eta_{\text{base}} = 0.00811012, \quad \eta_{\text{work}} = 0.01075298. \quad (31)$$

绝对提升为

$$\Delta\eta = \eta_{\text{work}} - \eta_{\text{base}} = 0.00264286, \quad (32)$$

相对提升为

$$\frac{\eta_{\text{work}} - \eta_{\text{base}}}{\eta_{\text{base}}} = 32.5872\%. \quad (33)$$

因此, 问题二的严格可行调节在相同离散和能量口径下改善了馈源接收性能。该结论描述的是题目给定几何和本文理想镜面假设下的相对变化, 不等同于包含衍射、遮挡和馈源方向图的实际天线效率。

八、灵敏度与稳健性分析

8.1 投影口径半径

工作口径的理论半径为 150 m, 但离散面板重心可能靠近边界。为判断边界选择是否改变结论, 在 16 阶细分下把投影口径半径取为 149.5、150.0 和 150.5 m。投影面积加权结果见表 6。

表 6 投影口径半径敏感性

半径/m	基准面接收比	工作面接收比	相对提升/%
149.5	0.00806220	0.01134351	40.70
150.0	0.00800792	0.01126826	40.71
150.5	0.00795501	0.01119322	40.71

半径每增加 0.5 m, 两表面的接收比均略有下降, 这是新增边缘微单元的反射落点更分散所致。但三个半径下的相对提升都约为 40.7%, 工作面优于基准面的方向不变。这里的提升率与 64 阶主结果不同, 是因为灵敏度实验固定使用 16 阶细分; 它只用于比较参数扰动, 不替代主结果。

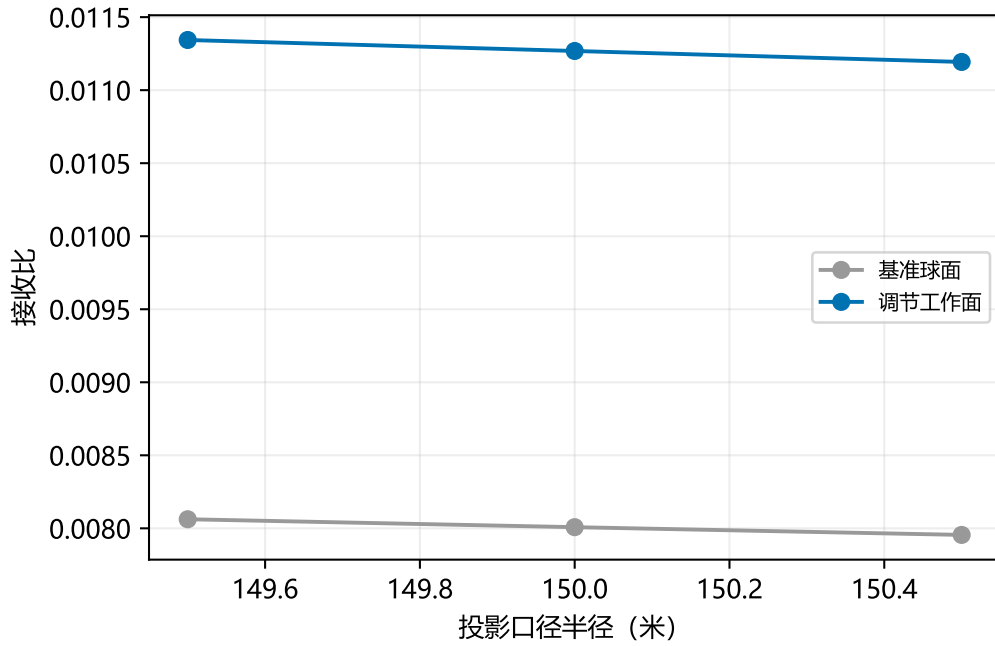


图 8 接收比对投影口径半径的敏感性

8.2 能量权重

将正式的投影面积权重改为面板等权后，149.5、150.0、150.5 m 三种半径下，基准面接收比分别变为 0.00802783、0.00797141、0.00791638，工作面分别为 0.01135302、0.01127372、0.01119453。相对于投影面积加权，各接收比的绝对变化均不超过约 4×10^{-5} ，且工作面仍优于基准面。这说明结论不依赖三角面板面积差异的具体权重处理。

8.3 优化可行性

顺序凸化从零位移、半缩放目标和最大可行缩放目标三个确定性起点运行。正式结果的最大促动器占用率为 0.415206，具有明显行程余量；最大主索占用率为 0.999988950，表明主索约束决定了最优形状。由于所有候选解都使用原始非线性几何逐项回代，求解器线性化误差不会被误认为物理可行性。

综合细分、口径和权重实验，可以确认“严格可行工作面提高接收比”这一结论具有稳定性；但主索约束接近活跃边界，若实际工程还存在未建模的安装或受力误差，应在控制应用中额外设置安全裕量。

九、模型评价与推广

9.1 模型优点

本文首先把理想面与可行工作面分层，避免用一个罚函数同时掩盖曲面定义和工程约束。焦点、口径、节点运动方向、下拉索长度和主索边均由统一坐标模型描述，三个子问题之间不存在符号或口径切换。

工作面求解使用严格可行起点、稀疏顺序凸化和原始非线性回代。促动器与主索约束逐项检查，求解器返回的“最优”状态不是可行性的唯一依据。二阶段 L_∞ 精修还限制了少数节点偏离理想面的最坏误差。

接收比模型使用有限三角面板而非连续解析曲面，并处理跨口径面板、投影面积权重和有限馈源圆盘。64 阶细分、口径半径扰动和面板等权对照为结果提供了可复查的数值稳定性证据。

9.2 模型局限

径向单自自由度假设没有建立索网力学平衡，也没有考虑预张力、弹性模量和节点切向位移。实际控制若需要同时保证索力安全，应把节点平衡方程、材料本构和促动器力限加入模型。

三角面板按三顶点平面处理，忽略面板柔性、拼缝、遮挡、衍射、馈源方向图和极化。因此本文接收比是题目几何光学假设下的相对指标，不应直接解释为 FAST 的实际电磁孔径效率。

顺序凸化求得的是多起点中的最佳局部可行解。虽然三个确定性起点得到稳定可行序列，本文没有给出全局最优证明。主索占用率接近 1，也意味着实际工程中需要为未建模误差保留额外裕量。

9.3 推广方向

该框架适用于“理想几何目标—局部执行器—稀疏相邻约束”类可重构结构，例如柔性索网天线、分块反射镜和可变形曲面。若获得索力与材料参数，可把顺序凸化扩展为形状误差和受力安全的多目标优化；若获得电磁模型，可在几何可行域内进一步优化增益、旁瓣和频带性能。

对于更大规模的离散曲面，主索雅可比仍保持局部稀疏，可以复用矩阵分解和热启动；射线追踪则可按面板批处理或并行化。无论采用何种扩展，原始约束回代与离散收敛检查仍应保留。

9.4 结论

固定焦点的一参数抛物面族使问题一具有唯一、可解释的选择变量；硬件友好焦距揭示主索比促动器更早成为限制。问题二在全部硬约束下得到 692 个节点的严格可行调节方案，最大主索相对变化 0.069999226%，最大促动器伸缩 0.249123 m。问题三的 64 阶面板射线追踪表明，工作面接收比相对基准球面提高 32.5872%。该提升在细分、口径和权重扰动下保持方向一致，说明约束可行化并未消除主动反射面调节的接收收益。

参考文献

- [1] 全国大学生数学建模竞赛组织委员会. 2021 年高教社杯全国大学生数学建模竞赛 A 题: FAST 主动反射面的形状调节[EB/OL]. 2021.
- [2] NAN R. Five hundred meter aperture spherical radio telescope (FAST)[J]. Science in China Series G: Physics, Mechanics and Astronomy, 2006, 49(2): 129–148. DOI: 10.1007/s11433-006-0129-9.
- [3] NAN R, LI D, JIN C, et al. The Five-Hundred-Meter Aperture Spherical Radio Telescope (FAST) Project[J]. International Journal of Modern Physics D, 2011, 20(6): 989–1024. DOI: 10.1142/S0218271811019335.
- [4] BOYD S, VANDENBERGHE L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004.
- [5] DIAMOND S, BOYD S. CVXPY: A Python-Embedded Modeling Language for Convex Optimization[J]. Journal of Machine Learning Research, 2016, 17(83): 1–5.
- [6] STELLATO B, BANJAC G, GOULART P, et al. OSQP: An Operator Splitting Solver for Quadratic Programs[J]. Mathematical Programming Computation, 2020, 12(4): 637–672. DOI: 10.1007/s12532-020-00179-2.

附录

附录 A：支撑材料文件清单

支撑材料建议包含以下内容：

1. code/：全部 Python 源程序；
2. tests/：正常、边界和失败输入测试；
3. data/raw/：题目附件的只读数值副本；
4. results/result.xlsx：题目要求的三工作表结果文件；
5. results/：正式优化、约束回代、射线收敛和敏感性表；
6. figures/：论文使用的数据图 PDF 与可编辑流程图源文件；
7. README.md：解释器、依赖和一键复现命令。

所有文件名、文件夹名和文档属性均不得包含参赛者、学校或赛区身份信息。

附录 B：完整源程序

以下代码与支撑材料中的源文件保持一致。为控制正文篇幅，程序只在附录出现。

几何、数据与约束模块

fast_geometry.py

```
"""FAST 数据读取与几何约束模块。

输入为附件 1--3 的 CSV 文件，主要输出节点、促动器、三角面板、
主索拓扑、观测方向和可精确回代的硬件约束指标。
"""

from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
from numpy.typing import NDArray

FloatArray = NDArray[np.float64]
IntArray = NDArray[np.int64]
BoolArray = NDArray[np.bool_]

NODE_COLUMNS = ["节点编号", "x坐标（米）", "y坐标（米）", "z坐标（米）"]
ACTUATOR_COLUMNS = [
    "对应主索节点编号",
    "下端点x坐标（米）",
    "下端点y坐标（米）",
    "下端点z坐标（米）",
    "基准态时上端点x坐标（米）",
    "基准态时上端点y坐标（米）",
    "基准态时上端点z坐标（米）",
]
PANEL_COLUMNS = ["主索节点1", "主索节点2", "主索节点3"]

class DataValidationError(ValueError):
    """表示附件字段、编号或数值不符合题面约定。"""
```

```

@dataclass(frozen=True)
class FastData:
    """保存 FAST 基准几何和由附件恢复的拓扑。"""

    node_ids: NDArray[np.str_]
    nodes: FloatArray
    actuator_lower: FloatArray
    actuator_upper: FloatArray
    panels: IntArray
    edges: IntArray
    radii: FloatArray
    node_units: FloatArray
    actuator_units: FloatArray
    down_cable_lengths: FloatArray
    edge_lengths: FloatArray

    @property
    def radius(self) -> float:
        """返回附件节点半径的算术均值，单位为米。"""

        return float(np.mean(self.radii))

@dataclass(frozen=True)
class DirectionFrame:
    """保存一个观测方向对应的局部轴线和口径掩码。"""

    sky: FloatArray
    axis: FloatArray
    focus: FloatArray
    active_nodes: BoolArray
    incident_edges: BoolArray

def _read_required_csv(path: Path, columns: list[str]) -> pd.DataFrame:
    try:
        frame = pd.read_csv(path, encoding="gb18030")
    except FileNotFoundError as exc:
        raise DataValidationError(f"缺少输入文件: {path}") from exc
    missing = [column for column in columns if column not in frame.columns]
    if missing:
        raise DataValidationError(f"{path.name} 缺失字段: {missing}")
    if frame[columns].isna().any().any():
        raise DataValidationError(f"{path.name} 存在缺失值")
    return frame[columns].copy()

def _validate_identifiers(node_frame: pd.DataFrame) -> NDArray[np.str_]:
    identifiers = node_frame[NODE_COLUMNS[0]].astype(str).str.strip()
    if identifiers.duplicated().any():
        duplicates = identifiers[identifiers.duplicated()].unique().tolist()
        raise DataValidationError(f"节点编号重复: {duplicates[:5]}")
    return identifiers.to_numpy(dtype=str)

def _map_panels(panel_frame: pd.DataFrame, index_by_id: dict[str, int]) -> IntArray:
    mapped = np.empty((len(panel_frame), 3), dtype=np.int64)
    for column_index, column in enumerate(PANEL_COLUMNS):
        values = panel_frame[column].astype(str).str.strip()
        unknown = sorted(set(values) - set(index_by_id))
        if unknown:
            raise DataValidationError(f"面板引用未知节点: {unknown[:5]}")

```

```

        mapped[:, column_index] = values.map(index_by_id).to_numpy(dtype=np.int64)
    if np.any(np.sort(mapped, axis=1)[: , 1:] == np.sort(mapped, axis=1)[: , :-1]):
        raise DataValidationError("面板存在重复顶点")
    return mapped

def _build_edges(panels: IntArray) -> IntArray:
    # 每块三角面板贡献三条边; 按无向端点排序去重, 恢复 6525 根主索。
    edge_set: set[tuple[int, int]] = set()
    for first, second, third in panels:
        edge_set.add(tuple(sorted((int(first), int(second)))))
        edge_set.add(tuple(sorted((int(second), int(third)))))
        edge_set.add(tuple(sorted((int(third), int(first)))))
    return np.asarray(sorted(edge_set), dtype=np.int64)

def load_fast_data(data_dir: Path) -> FastData:
    """读取并验证附件 1--3。

    参数:
        data_dir: 包含三个 GB18030 CSV 文件的目录。

    返回:
        可直接用于几何计算的不可变 ``FastData``。

    异常:
        DataValidationError: 字段、编号、形状或几何量不合法。
    """

    node_frame = _read_required_csv(data_dir / "附件1.csv", NODE_COLUMNS)
    node_ids = _validate_identifiers(node_frame)
    index_by_id = {identifier: index for index, identifier in enumerate(node_ids)}
    actuator_frame = _read_required_csv(data_dir / "附件2.csv", ACTUATOR_COLUMNS)
    panel_frame = _read_required_csv(data_dir / "附件3.csv", PANEL_COLUMNS)

    actuator_ids = actuator_frame[ACTUATOR_COLUMNS[0]].astype(str).str.strip()
    if actuator_ids.tolist() != node_ids.tolist():
        raise DataValidationError("附件2的节点顺序或编号与附件1不一致")
    panels = _map_panels(panel_frame, index_by_id)
    edges = _build_edges(panels)

    nodes = node_frame[NODE_COLUMNS[1:]].to_numpy(dtype=float)
    lower = actuator_frame[ACTUATOR_COLUMNS[1:4]].to_numpy(dtype=float)
    upper = actuator_frame[ACTUATOR_COLUMNS[4:7]].to_numpy(dtype=float)
    if nodes.shape != (2226, 3) or panels.shape != (4300, 3):
        raise DataValidationError("附件规模与题面给出的 2226 节点、4300 面板不一致")
    if edges.shape != (6525, 2):
        raise DataValidationError("三角拓扑未恢复出题面规定的 6525 条主索")

    radii = np.linalg.norm(nodes, axis=1)
    upper_radii = np.linalg.norm(upper, axis=1)
    if np.any(radii <= 0) or np.any(upper_radii <= 0):
        raise DataValidationError("节点或促动器顶端出现零半径")
    edge_vectors = nodes[edges[:, 0]] - nodes[edges[:, 1]]
    edge_lengths = np.linalg.norm(edge_vectors, axis=1)
    if np.any(edge_lengths <= 0):
        raise DataValidationError("主索边长度必须为正")

    return FastData(
        node_ids=node_ids,
        nodes=nodes,
        actuator_lower=lower,

```

```

        actuator_upper=upper,
        panels=panels,
        edges=edges,
        radii=radii,
        node_units=nodes / radii[:, None],
        actuator_units=upper / upper_radii[:, None],
        down_cable_lengths=np.linalg.norm(nodes - upper, axis=1),
        edge_lengths=edge_lengths,
    )

def build_direction_frame(
    data: FastData,
    alpha_deg: float,
    beta_deg: float,
    aperture_radius: float = 150.0,
) -> DirectionFrame:
    """构造观测轴、物理焦点及基准态工作口径。

    方位角和仰角使用度；口径半径单位为米。活动节点依据其在
    垂直观测轴平面上的投影距离选择。
    """

    if aperture_radius <= 0:
        raise ValueError("口径半径必须为正")
    alpha, beta = np.deg2rad([alpha_deg, beta_deg])
    sky = np.array(
        [
            np.cos(beta) * np.cos(alpha),
            np.cos(beta) * np.sin(alpha),
            np.sin(beta),
        ],
        dtype=float,
    )
    sky /= np.linalg.norm(sky)
    axis = -sky
    axial = data.nodes @ axis
    rho_squared = np.sum(data.nodes**2, axis=1) - axial**2
    active = (axial > 0) & (rho_squared <= aperture_radius**2)
    incident = active[data.edges[:, 0]] | active[data.edges[:, 1]]
    focus_radius = (1.0 - 0.466) * data.radius
    return DirectionFrame(
        sky=sky,
        axis=axis,
        focus=focus_radius * axis,
        active_nodes=active,
        incident_edges=incident,
    )

def paraboloid_target_displacement(
    data: FastData,
    frame: DirectionFrame,
    focal_length: float,
) -> FloatArray:
    """计算节点径向射线与指定理想抛物面的交点位移。

    返回与全部节点等长的数组，口径外节点位移为零；正值表示
    节点朝球心移动。焦距和位移单位均为米。
    """

    if focal_length <= 0:

```

```

        raise ValueError("抛物面焦距必须为正")
    focus_radius = float(np.linalg.norm(frame.focus))
    vertex_radius = focus_radius + focal_length
    cosine = data.node_units @ frame.axis
    radial_factor = 1.0 - cosine**2
    root_term = focal_length**2 * cosine**2
    root_term += focal_length * vertex_radius * radial_factor
    roots = -2.0 * focal_length * cosine + 2.0 * np.sqrt(root_term)
    intersection = np.divide(
        roots,
        radial_factor,
        out=np.zeros_like(roots),
        where=np.abs(radial_factor) > 1e-12,
    )
    axial_case = vertex_radius / cosine
    intersection = np.where(np.abs(radial_factor) <= 1e-12, axial_case, intersection)
    displacement = np.zeros_like(data.radii)
    displacement[frame.active_nodes] = (
        data.radii[frame.active_nodes] - intersection[frame.active_nodes]
    )
    return displacement

def actuator_strokes(
    data: FastData,
    displacement: FloatArray,
) -> tuple[FloatArray, BoolArray]:
    """由节点径向位移求保持下拉索定长的促动器伸缩量。

    返回伸缩量和逐节点可达标记。选择在零位移处等于零并连续的
    二次方程根，正伸缩量表示促动器顶端向球心运动。
    """

    if displacement.shape != data.radii.shape:
        raise ValueError("位移数组形状错误")
    delta = data.nodes - data.actuator_upper
    moved_delta = delta - displacement[:, None] * data.node_units
    # 下拉索定长方程关于促动器伸缩量是一元二次式，必须沿零位移根连续取支。
    coefficient = np.sum(data.actuator_units * moved_delta, axis=1)
    constant = np.sum(moved_delta**2, axis=1) - data.down_cable_lengths**2
    discriminant = coefficient**2 - constant
    reachable = discriminant >= -1e-10
    safe_discriminant = np.maximum(discriminant, 0.0)
    baseline_coefficient = np.sum(data.actuator_units * delta, axis=1)
    branch = np.where(baseline_coefficient >= 0.0, 1.0, -1.0)
    strokes = -coefficient + branch * np.sqrt(safe_discriminant)
    strokes[~reachable] = np.nan
    return strokes, reachable

def constraint_summary(
    data: FastData,
    frame: DirectionFrame,
    displacement: FloatArray,
) -> dict[str, float | int | bool]:
    """精确回代促动器和主索约束并返回最坏指标。"""

    strokes, reachable = actuator_strokes(data, displacement)
    moved_nodes = data.nodes - displacement[:, None] * data.node_units
    edges = data.edges[frame.incident_edges]
    baseline = data.edge_lengths[frame.incident_edges]
    moved_edge_vectors = moved_nodes[edges[:, 0]] - moved_nodes[edges[:, 1]]

```

```

        relative_changes = np.abs(np.linalg.norm(moved_edge_vectors, axis=1) / baseline -
1.0)
        active_strokes = strokes[frame.active_nodes]
        active_reachable = reachable[frame.active_nodes]
        max_stroke = float(np.nanmax(np.abs(active_strokes)))
        max_edge_change = float(np.max(relative_changes))
        return {
            "active_nodes": int(np.sum(frame.active_nodes)),
            "incident_edges": int(np.sum(frame.incident_edges)),
            "reachable": bool(np.all(active_reachable)),
            "max_abs_stroke_m": max_stroke,
            "max_edge_relative_change": max_edge_change,
            "actuator_utilization": max_stroke / 0.6,
            "edge_utilization": max_edge_change / 0.0007,
            "feasible": bool(
                np.all(active_reachable)
                and max_stroke <= 0.6 + 1e-10
                and max_edge_change <= 0.0007 + 1e-12
            ),
        }

def hardware_utilization(
    data: FastData,
    frame: DirectionFrame,
    focal_length: float,
) -> float:
    """返回理想目标面的最坏归一化硬件占用率。"""

    displacement = paraboloid_target_displacement(data, frame, focal_length)
    summary = constraint_summary(data, frame, displacement)
    if not summary["reachable"]:
        return float("inf")
    return float(max(summary["actuator_utilization"], summary["edge_utilization"]))

def node_area_weights(data: FastData) -> FloatArray:
    """按相邻三角面板面积的三分之一构造节点面积权重。"""

    vertices = data.nodes[data.panels]
    cross = np.cross(vertices[:, 1] - vertices[:, 0], vertices[:, 2] - vertices[:, 0])
    areas = 0.5 * np.linalg.norm(cross, axis=1)
    weights = np.zeros(len(data.node_ids), dtype=float)
    for corner in range(3):
        np.add.at(weights, data.panels[:, corner], areas / 3.0)
    if np.any(weights <= 0):
        raise DataValidationError("存在未获得正面积权重的节点")
    return weights

```

问题一至问题三基础模块

problem1.py

```

"""问题一和问题二共享的理想抛物面一维搜索。

输入为已验证的 FAST 几何和观测方向，输出最小硬件占用率对应的
焦距、目标位移、约束摘要以及可审计的搜索网格。
"""

from __future__ import annotations

from dataclasses import dataclass

```

```

import numpy as np
from numpy.typing import NDArray
from scipy.optimize import minimize_scalar

from fast_geometry import (
    DirectionFrame,
    FastData,
    constraint_summary,
    hardware_utilization,
    node_area_weights,
    paraboloid_target_displacement,
)

FloatArray = NDArray[np.float64]

@dataclass(frozen=True)
class FocalSearchResult:
    """保存 G2 一维焦距搜索的结果与诊断网格。"""

    focal_length: float
    displacement: FloatArray
    constraint: dict[str, float | int | bool]
    grid_focal_lengths: FloatArray
    grid_utilizations: FloatArray
    optimizer_success: bool
    optimizer_message: str

@dataclass(frozen=True)
class FocalModelComparison:
    """保存几何基线、最小二乘和硬件友好三种理想面。"""

    geometric_focal_length: float
    least_squares_focal_length: float
    hardware_result: FocalSearchResult
    metrics: dict[str, dict[str, float | int | bool]]

def search_hardware_friendly_focal_length(
    data: FastData,
    frame: DirectionFrame,
    half_width: float = 0.8,
    grid_size: int = 65,
) -> FocalSearchResult:
    """执行 G2 有界网格加局部精化的最小焦距搜索。

    参数:
        data: FAST 基准几何。
        frame: 当前观测方向。
        half_width: 以 ``F=0.466R`` 为中心的 G2 搜索半宽，单位米。
        grid_size: 确定性搜索网格点数。

    返回:
        搜索结果、目标位移和精确约束回代摘要。
    """

    if half_width <= 0 or grid_size < 5:
        raise ValueError("搜索半宽必须为正且网格点数至少为 5")
    baseline_focal = 0.466 * data.radius
    grid = np.linspace(

```

```

        baseline_focal - half_width,
        baseline_focal + half_width,
        grid_size,
    )
    utilization = np.array(
        [hardware_utilization(data, frame, value) for value in grid],
        dtype=float,
    )
    best_index = int(np.argmin(utilization))
    left_index = max(0, best_index - 1)
    right_index = min(grid_size - 1, best_index + 1)

    # G2 先用一维有界精化验证目标可计算; G3 再执行自动扩张与完整对照。
    optimized = minimize_scalar(
        lambda value: hardware_utilization(data, frame, float(value)),
        bounds=(float(grid[left_index]), float(grid[right_index])),
        method="bounded",
        options={"xatol": 1e-10, "maxiter": 200},
    )
    focal_length = float(optimized.x if optimized.success else grid[best_index])
    displacement = paraboloid_target_displacement(data, frame, focal_length)
    return FocalSearchResult(
        focal_length=focal_length,
        displacement=displacement,
        constraint=constraint_summary(data, frame, displacement),
        grid_focal_lengths=grid,
        grid_utilizations=utilization,
        optimizer_success=bool(optimized.success),
        optimizer_message=str(optimized.message),
    )

def _weighted_movement_error(
    data: FastData,
    frame: DirectionFrame,
    focal_length: float,
) -> float:
    displacement = paraboloid_target_displacement(data, frame, focal_length)
    active = frame.active_nodes
    weights = node_area_weights(data)[active]
    return float(np.average(displacement[active] ** 2, weights=weights))

def compare_focal_models(
    data: FastData,
    frame: DirectionFrame,
    initial_half_width: float = 0.2,
) -> FocalModelComparison:
    """比较 G1 批准的三种理想抛物面并自动扩张搜索区间。"""

    if initial_half_width <= 0:
        raise ValueError("初始搜索半宽必须为正")
    half_width = initial_half_width
    baseline_focal = 0.466 * data.radius
    hardware = search_hardware_friendly_focal_length(
        data,
        frame,
        half_width=half_width,
        grid_size=65,
    )
    while half_width < 3.2:
        grid = hardware.grid_utilizations
        boundary_is_competitive = min(grid[0], grid[-1]) <= 1.05 * float(np.min(grid))

```

```

        if not boundary_is_competitive:
            break
        half_width *= 2.0
        hardware = search_hardware_friendly_focal_length(
            data,
            frame,
            half_width=half_width,
            grid_size=129,
        )

    bounds = (baseline_focal - half_width, baseline_focal + half_width)
    least_squares = minimize_scalar(
        lambda value: _weighted_movement_error(data, frame, float(value)),
        bounds=bounds,
        method="bounded",
        options={"xatol": 1e-10, "maxiter": 300},
    )
    if not least_squares.success:
        raise RuntimeError(f"最小二乘焦距搜索失败: {least_squares.message}")

    candidates = {
        "geometric": baseline_focal,
        "least_squares": float(least_squares.x),
        "hardware_friendly": hardware.focal_length,
    }
    metrics: dict[str, dict[str, float | int | bool]] = {}
    for name, focal_length in candidates.items():
        displacement = paraboloid_target_displacement(data, frame, focal_length)
        summary = constraint_summary(data, frame, displacement)
        summary["movement_mse_m2"] = _weighted_movement_error(
            data,
            frame,
            focal_length,
        )
        metrics[name] = summary
    return FocalModelComparison(
        geometric_focal_length=baseline_focal,
        least_squares_focal_length=float(least_squares.x),
        hardware_result=hardware,
        metrics=metrics,
    )

```

problem2.py

```

"""问题二最小可行调节基线。

G2 只验证理想目标是否已经满足硬约束；若不满足，则回到严格可行的
基准球面。正式顺序凸化优化留待 G2 批准后的 G3。
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from numpy.typing import NDArray

from fast_geometry import DirectionFrame, FastData, constraint_summary

FloatArray = NDArray[np.float64]

```

```

@dataclass(frozen=True)
class FeasibleBaseline:
    """保存 G2 选择的可行位移及其来源。"""

    displacement: FloatArray
    source: str
    constraint: dict[str, float | int | bool]
    weighted_mean_square_error: float
    max_abs_target_error_m: float

def choose_feasible_baseline(
    data: FastData,
    frame: DirectionFrame,
    target_displacement: FloatArray,
) -> FeasibleBaseline:
    """选择理想目标或零位移作为 G2 可行基线。

    理想目标通过精确约束回代时直接采用；否则使用基准球面零位移。
    返回值只证明可行性，不代表 G3 的正式最优调节结果。
    """

    target_constraint = constraint_summary(data, frame, target_displacement)
    if target_constraint["feasible"]:
        selected = target_displacement.copy()
        source = "ideal-target-feasible"
        selected_constraint = target_constraint
    else:
        # 零位移保持全部基准几何不变，是目标投影不可行时的确定性安全基线。
        selected = np.zeros_like(target_displacement)
        source = "zero-displacement-feasible"
        selected_constraint = constraint_summary(data, frame, selected)
    errors = selected[frame.active_nodes] - target_displacement[frame.active_nodes]
    return FeasibleBaseline(
        displacement=selected,
        source=source,
        constraint=selected_constraint,
        weighted_mean_square_error=float(np.mean(errors**2)),
        max_abs_target_error_m=float(np.max(np.abs(errors))),
    )

```

problem3.py

```

"""问题三的反射定律与馈源命中判据。

G2 通过解析理想抛物面聚焦残差验证符号、法向和馈源平面求交；
完整三角面板细分及正式接收比留待 G3。
"""

from __future__ import annotations

import numpy as np
from numpy.typing import NDArray

from fast_geometry import DirectionFrame

FloatArray = NDArray[np.float64]
BoolArray = NDArray[np.bool_]

```

```

def reflect_directions(incident: FloatArray, normals: FloatArray) -> FloatArray:
    """按镜面反射定律计算单位反射方向。"""

    if incident.shape != normals.shape or incident.ndim != 2 or incident.shape[1] != 3:
        raise ValueError("入射方向和法向必须是相同形状的 N×3 数组")
    unit_incident = incident / np.linalg.norm(incident, axis=1)[:, None]
    unit_normals = normals / np.linalg.norm(normals, axis=1)[:, None]
    # 镜面反射只改变法向分量, 切向分量保持不变。
    projections = np.sum(unit_incident * unit_normals, axis=1)
    reflected = unit_incident - 2.0 * projections[:, None] * unit_normals
    return reflected / np.linalg.norm(reflected, axis=1)[:, None]

def receiver_hits(
    points: FloatArray,
    directions: FloatArray,
    focus: FloatArray,
    sky: FloatArray,
    receiver_radius: float = 0.5,
) -> BoolArray:
    """判断反射线是否正向命中馈源平面的有效圆盘。"""

    if points.shape != directions.shape or points.ndim != 2 or points.shape[1] != 3:
        raise ValueError("射线起点和方向必须是相同形状的 N×3 数组")
    if receiver_radius <= 0:
        raise ValueError("馈源有效半径必须为正")
    denominator = directions @ sky
    numerator = (focus[None, :] - points) @ sky
    valid = np.abs(denominator) > 1e-14
    distance = np.full(len(points), np.nan, dtype=float)
    distance[valid] = numerator[valid] / denominator[valid]
    intersections = points + distance[:, None] * directions
    radial_distance = np.linalg.norm(intersections - focus[None, :], axis=1)
    return valid & (distance > 0) & (radial_distance <= receiver_radius)

def ideal_paraboloid_focus_error(
    frame: DirectionFrame,
    focal_length: float,
    aperture_radius: float = 150.0,
) -> float:
    """计算解析理想抛物面测试射线到焦点的最大残差, 单位米。"""

    if focal_length <= 0 or aperture_radius <= 0:
        raise ValueError("焦距和口径半径必须为正")
    reference = np.array([1.0, 0.0, 0.0])
    if abs(float(reference @ frame.axis)) > 0.9:
        reference = np.array([0.0, 1.0, 0.0])
    first = np.cross(frame.axis, reference)
    first /= np.linalg.norm(first)
    second = np.cross(frame.axis, first)
    radii = np.linspace(0.0, aperture_radius, 7)
    angles = np.linspace(0.0, 2.0 * np.pi, 13, endpoint=False)
    rho, angle = np.meshgrid(radii, angles, indexing="ij")
    transverse = np.cos(angle)[..., None] * first
    transverse += np.sin(angle)[..., None] * second
    focus_radius = float(np.linalg.norm(frame.focus))
    vertex_radius = focus_radius + focal_length
    axial = vertex_radius - rho**2 / (4.0 * focal_length)
    points = axial[..., None] * frame.axis + rho[..., None] * transverse
    gradient = 2.0 * rho[..., None] * transverse + 4.0 * focal_length * frame.axis
    normals = -gradient / np.linalg.norm(gradient, axis=2)[..., None]

```

```

flat_points = points.reshape(-1, 3)
flat_normals = normals.reshape(-1, 3)
incident = np.repeat(frame.axis[None, :], len(flat_points), axis=0)
reflected = reflect_directions(incident, flat_normals)
line_parameter = np.sum(
    (frame.focus[None, :] - flat_points) * reflected,
    axis=1,
)
closest = flat_points + line_parameter[:, None] * reflected
return float(np.max(np.linalg.norm(closest - frame.focus[None, :], axis=1)))

```

正式约束优化与射线追踪

optimization.py

"""FAST 工作反射面的正式约束优化。

模块使用 CVXPY 的稀疏顺序凸化子问题，并在每次候选更新后用原始非线性几何精确回代。输入为目标径向位移，输出严格可行的节点位移。

```

from __future__ import annotations

from dataclasses import dataclass
from typing import Any

import cvxpy as cp
import numpy as np
from numpy.typing import NDArray
from scipy import sparse

from fast_geometry import (
    DirectionFrame,
    FastData,
    actuator_strokes,
    constraint_summary,
    node_area_weights,
)

FloatArray = NDArray[np.float64]

@dataclass(frozen=True)
class OptimizationResult:
    """保存正式优化解、精确约束摘要和迭代证据。"""

    displacement: FloatArray
    strokes: FloatArray
    adjusted_nodes: FloatArray
    objective_mse_m2: float
    max_abs_error_m: float
    constraint: dict[str, float | int | bool]
    selected_start: str
    histories: list[dict[str, Any]]
    solver_statuses: list[str]

def _full_displacement(
    frame: DirectionFrame,
    active_values: FloatArray,
) -> FloatArray:
    full = np.zeros(len(frame.active_nodes), dtype=float)

```

```

full[frame.active_nodes] = active_values
return full

def _objective(
    active_values: FloatArray,
    target: FloatArray,
    weights: FloatArray,
) -> float:
    return float(np.average((active_values - target) ** 2, weights=weights))

def _edge_linearization(
    data: FastData,
    frame: DirectionFrame,
    current_full: FloatArray,
    active_index: NDArray[np.int64],
) -> tuple[FloatArray, sparse.csr_matrix]:
    moved = data.nodes - current_full[:, None] * data.node_units
    edges = data.edges[frame.incident_edges]
    differences = moved[edges[:, 0]] - moved[edges[:, 1]]
    lengths = np.linalg.norm(differences, axis=1)
    index_map = np.full(len(data.node_ids), -1, dtype=np.int64)
    index_map[active_index] = np.arange(len(active_index))
    rows: list[int] = []
    columns: list[int] = []
    values: list[float] = []
    for row, (left, right) in enumerate(edges):
        left_column = int(index_map[left])
        right_column = int(index_map[right])
        if left_column >= 0:
            derivative = -float(differences[row] @ data.node_units[left]) /
lengths[row]
            rows.append(row)
            columns.append(left_column)
            values.append(derivative)
        if right_column >= 0:
            derivative = float(differences[row] @ data.node_units[right]) /
lengths[row]
            rows.append(row)
            columns.append(right_column)
            values.append(derivative)
    jacobian = sparse.coo_matrix(
        (values, (rows, columns)),
        shape=(len(edges), len(active_index)),
    ).tocsr()
    return lengths, jacobian

def _stroke_linearization(
    data: FastData,
    current_full: FloatArray,
    active_index: NDArray[np.int64],
) -> tuple[FloatArray, FloatArray]:
    strokes, reachable = actuator_strokes(data, current_full)
    if not np.all(reachable[active_index]):
        raise RuntimeError("当前迭代点出现不可达下拉索几何")
    delta = data.nodes - data.actuator_upper
    cable_vectors = delta - current_full[:, None] * data.node_units
    cable_vectors += strokes[:, None] * data.actuator_units
    numerator = np.sum(cable_vectors * data.node_units, axis=1)
    denominator = np.sum(cable_vectors * data.actuator_units, axis=1)
    if np.any(np.abs(denominator[active_index]) < 1e-10):

```

```

        raise RuntimeError("促动器连续根导数出现奇异分母")
    derivative = numerator / denominator
    return strokes[active_index], derivative[active_index]

def _is_strictly_feasible(
    data: FastData,
    frame: DirectionFrame,
    full_displacement: FloatArray,
) -> bool:
    summary = constraint_summary(data, frame, full_displacement)
    return bool(summary["feasible"])

def maximum_feasible_target_scale(
    data: FastData,
    frame: DirectionFrame,
    target_displacement: FloatArray,
    iterations: int = 60,
) -> float:
    """二分求目标位移射线上仍满足全部精确硬约束的最大比例。"""

    if iterations < 1:
        raise ValueError("二分迭代次数必须为正")
    if _is_strictly_feasible(data, frame, target_displacement):
        return 1.0
    lower, upper = 0.0, 1.0
    for _ in range(iterations):
        middle = 0.5 * (lower + upper)
        if _is_strictly_feasible(data, frame, middle * target_displacement):
            lower = middle
        else:
            upper = middle
    return lower

def _solve_l2_subproblem(
    data: FastData,
    frame: DirectionFrame,
    current: FloatArray,
    target: FloatArray,
    weights: FloatArray,
    trust_radius: float,
) -> tuple[FloatArray, str]:
    active_index = np.flatnonzero(frame.active_nodes)
    current_full = _full_displacement(frame, current)
    lengths, edge_jacobian = _edge_linearization(
        data,
        frame,
        current_full,
        active_index,
    )
    strokes, stroke_derivative = _stroke_linearization(
        data,
        current_full,
        active_index,
    )
    variable = cp.Variable(len(active_index))
    delta = variable - current
    baseline_edges = data.edge_lengths[frame.incident_edges]
    linear_edges = lengths + edge_jacobian @ delta
    linear_strokes = strokes + cp.multiply(stroke_derivative, delta)
    constraints = [

```

```

        linear_edges >= baseline_edges * (1.0 - 0.0007) + 1e-7,
        linear_edges <= baseline_edges * (1.0 + 0.0007) - 1e-7,
        linear_strokes >= -0.6 + 1e-7,
        linear_strokes <= 0.6 - 1e-7,
        cp.abs(delta) <= trust_radius,
    ]
    normalized_weights = weights / np.sum(weights)
    objective = cp.Minimize(cp.sum(cp.multiply(normalized_weights, cp.square(variable -
target))))
    problem = cp.Problem(objective, constraints)
    problem.solve(
        solver=cp.OSQP,
        eps_abs=1e-10,
        eps_rel=1e-9,
        max_iter=200000,
        polishing=True,
        verbose=False,
    )
    if problem.status not in {cp.OPTIMAL, cp.OPTIMAL_INACCURATE}:
        raise RuntimeError(f"OSQP 子问题失败: {problem.status}")
    return np.asarray(variable.value, dtype=float), str(problem.status)

def _accept_feasible_step(
    data: FastData,
    frame: DirectionFrame,
    current: FloatArray,
    candidate: FloatArray,
    target: FloatArray,
    weights: FloatArray,
) -> tuple[FloatArray, float]:
    current_objective = _objective(current, target, weights)
    for exponent in range(18):
        fraction = 0.5**exponent
        trial = current + fraction * (candidate - current)
        full_trial = _full_displacement(frame, trial)
        if _is_strictly_feasible(data, frame, full_trial):
            if _objective(trial, target, weights) < current_objective - 1e-15:
                return trial, fraction
    return current.copy(), 0.0

def _accept_linf_step(
    data: FastData,
    frame: DirectionFrame,
    current: FloatArray,
    candidate: FloatArray,
    target: FloatArray,
    weights: FloatArray,
    l2_limit: float,
) -> tuple[FloatArray, float]:
    """在不突破 L2 容差时，回溯接受能降低最大绝对误差的精修步。"""

    current_max = float(np.max(np.abs(current - target)))
    for exponent in range(18):
        fraction = 0.5**exponent
        trial = current + fraction * (candidate - current)
        full_trial = _full_displacement(frame, trial)
        trial_max = float(np.max(np.abs(trial - target)))
        if (
            _is_strictly_feasible(data, frame, full_trial)
            and _objective(trial, target, weights) <= l2_limit + 1e-12
            and trial_max < current_max - 1e-12

```

```

    ):
        return trial, fraction
    return current.copy(), 0.0

def _run_l2_scp(
    data: FastData,
    frame: DirectionFrame,
    target: FloatArray,
    weights: FloatArray,
    initial: FloatArray,
    start_name: str,
    max_iterations: int,
) -> tuple[FloatArray, list[dict[str, Any]], list[str]]:
    current = initial.copy()
    trust_radius = 0.08
    history: list[dict[str, Any]] = []
    statuses: list[str] = []
    for iteration in range(max_iterations):
        candidate, status = _solve_l2_subproblem(
            data,
            frame,
            current,
            target,
            weights,
            trust_radius,
        )
        statuses.append(status)
        accepted, fraction = _accept_feasible_step(
            data,
            frame,
            current,
            candidate,
            target,
            weights,
        )
        step = float(np.max(np.abs(accepted - current)))
        previous = _objective(current, target, weights)
        current = accepted
        current_objective = _objective(current, target, weights)
        history.append(
            {
                "start": start_name,
                "stage": "l2",
                "iteration": iteration,
                "objective_mse_m2": current_objective,
                "max_step_m": step,
                "accepted_fraction": fraction,
                "trust_radius_m": trust_radius,
            }
        )
        if fraction == 1.0:
            trust_radius = min(0.2, 1.4 * trust_radius)
        elif fraction > 0:
            trust_radius = max(1e-4, 0.6 * trust_radius)
        else:
            trust_radius = max(1e-4, 0.35 * trust_radius)
        improvement = previous - current_objective
        if step < 1e-7 or improvement <= 1e-12 * max(1.0, previous):
            break
    return current, history, statuses

```

```

def _refine_linf(
    data: FastData,
    frame: DirectionFrame,
    target: FloatArray,
    weights: FloatArray,
    initial: FloatArray,
    l2_limit: float,
    max_iterations: int = 12,
) -> tuple[FloatArray, list[dict[str, Any]], list[str]]:
    current = initial.copy()
    active_index = np.flatnonzero(frame.active_nodes)
    normalized_weights = weights / np.sum(weights)
    trust_radius = 0.03
    history: list[dict[str, Any]] = []
    statuses: list[str] = []
    for iteration in range(max_iterations):
        current_full = _full_displacement(frame, current)
        lengths, edge_jacobian = _edge_linearization(
            data,
            frame,
            current_full,
            active_index,
        )
        strokes, stroke_derivative = _stroke_linearization(
            data,
            current_full,
            active_index,
        )
        variable = cp.Variable(len(active_index))
        maximum_error = cp.Variable(nonneg=True)
        delta = variable - current
        baseline_edges = data.edge_lengths[frame.incident_edges]
        constraints = [
            lengths + edge_jacobian @ delta >= baseline_edges * (1.0 - 0.0007) + 1e-7,
            lengths + edge_jacobian @ delta <= baseline_edges * (1.0 + 0.0007) - 1e-7,
            strokes + cp.multiply(stroke_derivative, delta) >= -0.6 + 1e-7,
            strokes + cp.multiply(stroke_derivative, delta) <= 0.6 - 1e-7,
            cp.abs(delta) <= trust_radius,
            cp.abs(variable - target) <= maximum_error,
            cp.sum(cp.multiply(normalized_weights, cp.square(variable - target))) <=
l2_limit,
        ]
        problem = cp.Problem(cp.Minimize(maximum_error), constraints)
        problem.solve(solver=cp.CLARABEL, max_iter=300, verbose=False)
        statuses.append(str(problem.status))
        if problem.status not in {cp.OPTIMAL, cp.OPTIMAL_INACCURATE}:
            break
        candidate = np.asarray(variable.value, dtype=float)
        previous_max = float(np.max(np.abs(current - target)))
        # L $\infty$  阶段允许均方误差在既定容差内轻微变化, 但必须改善最坏节点。
        accepted, fraction = _accept_linf_step(
            data,
            frame,
            current,
            candidate,
            target,
            weights,
            l2_limit,
        )
        step = float(np.max(np.abs(accepted - current)))
        current = accepted
        current_max = float(np.max(np.abs(current - target)))
        history.append(

```

```

        {
            "start": "selected-l2",
            "stage": "linf",
            "iteration": iteration,
            "objective_mse_m2": _objective(current, target, weights),
            "max_abs_error_m": current_max,
            "max_step_m": step,
            "accepted_fraction": fraction,
        }
    )
    if step < 1e-7 or previous_max - current_max < 1e-10:
        break
    return current, history, statuses

def optimize_work_surface(
    data: FastData,
    frame: DirectionFrame,
    target_displacement: FloatArray,
    max_iterations: int = 35,
) -> OptimizationResult:
    """执行多起点 L2 顺序凸化和近等价  $L^\infty$  二阶段精修。"""

    active_index = np.flatnonzero(frame.active_nodes)
    target = target_displacement[active_index]
    weights = node_area_weights(data)[active_index]
    scale = maximum_feasible_target_scale(data, frame, target_displacement)
    starts = {
        "zero": np.zeros_like(target),
        "half-scaled-target": 0.5 * scale * target,
        "max-scaled-target": scale * target,
    }
    candidates: list[tuple[str, FloatArray, list[dict[str, Any]], list[str]]] = []
    for name, initial in starts.items():
        solved, history, statuses = _run_l2_scp(
            data,
            frame,
            target,
            weights,
            initial,
            name,
            max_iterations,
        )
        candidates.append((name, solved, history, statuses))
    selected = min(candidates, key=lambda item: _objective(item[1], target, weights))
    selected_name, best_l2, history, statuses = selected
    l2_objective = _objective(best_l2, target, weights)
    l2_limit = l2_objective * (1.0 + 1e-6) + 1e-12
    refined, linf_history, linf_statuses = _refine_linf(
        data,
        frame,
        target,
        weights,
        best_l2,
        l2_limit,
    )
    full = _full_displacement(frame, refined)
    strokes, reachable = actuator_strokes(data, full)
    if not np.all(reachable[active_index]):
        raise RuntimeError("正式解存在不可达促动器")
    summary = constraint_summary(data, frame, full)
    if not summary["feasible"]:
        raise RuntimeError(f"正式解未通过精确硬约束: {summary}")

```

```

return OptimizationResult(
    displacement=full,
    strokes=strokes,
    adjusted_nodes=data.nodes - full[:, None] * data.node_units,
    objective_mse_m2=_objective(refined, target, weights),
    max_abs_error_m=float(np.max(np.abs(refined - target))),
    constraint=summary,
    selected_start=selected_name,
    histories=[entry for item in candidates for entry in item[2]] + linf_history,
    solver_statuses=[status for item in candidates for status in item[3]] +
    linf_statuses,
)

```

raytrace.py

```

"""FAST 三角面板确定性细分射线追踪。

输入为基准或调整后的节点坐标，输出 300 米投影口径内的投影面积
加权接收比、收敛记录和面板中心在馈源平面的落点。
"""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
from numpy.typing import NDArray

from fast_geometry import DirectionFrame, FastData
from problem3 import reflect_directions

FloatArray = NDArray[np.float64]

@dataclass(frozen=True)
class RayTraceResult:
    """保存一个细分级别的接收比和能量统计。"""

    subdivision: int
    reception_ratio: float
    received_weight: float
    total_weight: float
    included_samples: int

def barycentric_micro_centroids(subdivision: int) -> FloatArray:
    """生成等面积三角细分的全部微三角形重心坐标。"""

    if subdivision < 1:
        raise ValueError("三角细分级别必须为正整数")
    coordinates: list[tuple[float, float, float]] = []
    scale = float(subdivision)
    for first in range(subdivision):
        for second in range(subdivision - first):
            beta = (first + 1.0 / 3.0) / scale
            gamma = (second + 1.0 / 3.0) / scale
            coordinates.append((1.0 - beta - gamma, beta, gamma))
            if first + second <= subdivision - 2:
                beta = (first + 2.0 / 3.0) / scale
                gamma = (second + 2.0 / 3.0) / scale
                coordinates.append((1.0 - beta - gamma, beta, gamma))

```

```

result = np.asarray(coordinates, dtype=float)
if result.shape != (subdivision**2, 3):
    raise RuntimeError("三角细分重心数量不等于 subdivision^2")
return result

def _face_geometry(
    data: FastData,
    frame: DirectionFrame,
    nodes: FloatArray,
) -> tuple[FloatArray, FloatArray, FloatArray]:
    vertices = nodes[data.panels]
    cross = np.cross(vertices[:, 1] - vertices[:, 0], vertices[:, 2] - vertices[:, 0])
    double_area = np.linalg.norm(cross, axis=1)
    if np.any(double_area <= 0):
        raise ValueError("调整后表面存在零面积面板")
    normals = cross / double_area[:, None]
    # 面板顶点顺序不保证统一, 法向一律翻到天体侧, 确保入射点积为负。
    flip = (normals @ frame.sky) < 0
    normals[flip] *= -1.0
    reflected = reflect_directions(
        np.repeat(frame.axis[None, :], len(normals), axis=0),
        normals,
    )
    return vertices, normals, reflected

def trace_surface(
    data: FastData,
    frame: DirectionFrame,
    nodes: FloatArray,
    subdivision: int,
    aperture_radius: float = 150.0,
    projected_weight: bool = True,
    batch_size: int = 256,
) -> RayTraceResult:
    """计算给定三角面板表面的确定性细分接收比。"""

    if nodes.shape != data.nodes.shape:
        raise ValueError("节点坐标形状必须与附件1一致")
    if aperture_radius <= 0 or batch_size < 1:
        raise ValueError("口径半径和批大小必须为正")
    barycentric = barycentric_micro_centroids(subdivision)
    vertices, normals, reflected = _face_geometry(data, frame, nodes)
    face_areas = 0.5 * np.linalg.norm(
        np.cross(vertices[:, 1] - vertices[:, 0], vertices[:, 2] - vertices[:, 0]),
        axis=1,
    )
    total_weight = 0.0
    received_weight = 0.0
    included_samples = 0
    for start in range(0, len(vertices), batch_size):
        stop = min(len(vertices), start + batch_size)
        points = np.einsum(
            "sk,bkd->bsd",
            barycentric,
            vertices[start:stop],
            optimize=True,
        )
        axial = np.einsum("bsd,d->bs", points, frame.axis)
        radius_squared = np.sum(points**2, axis=2) - axial**2
        in_aperture = (axial > 0) & (radius_squared <= aperture_radius**2)

```

```

        directions = reflected[start:stop]
        denominator = directions @ frame.sky
        numerator = np.einsum(
            "bsd,d->bs",
            frame.focus[None, None, :] - points,
            frame.sky,
        )
        forward = np.abs(denominator) > 1e-14
        distance = np.divide(
            numerator,
            denominator[:, None],
            out=np.full_like(numerator, np.nan),
            where=forward[:, None],
        )
        intersections = points + distance[..., None] * directions[:, None, :]
        receiver_distance = np.linalg.norm(
            intersections - frame.focus[None, None, :],
            axis=2,
        )
        hit = in_aperture & (distance > 0) & (receiver_distance <= 0.5)
        if projected_weight:
            face_weight = face_areas[start:stop]
            face_weight *= np.abs(normals[start:stop] @ frame.axis)
        else:
            face_weight = np.ones(stop - start, dtype=float)
            sample_weight = face_weight[:, None] / float(subdivision**2)
            total_weight += float(np.sum(sample_weight * in_aperture))
            received_weight += float(np.sum(sample_weight * hit))
            included_samples += int(np.sum(in_aperture))
        ratio = received_weight / total_weight
        return RayTraceResult(
            subdivision=subdivision,
            reception_ratio=ratio,
            received_weight=received_weight,
            total_weight=total_weight,
            included_samples=included_samples,
        )

def trace_convergence(
    data: FastData,
    frame: DirectionFrame,
    nodes: FloatArray,
    levels: tuple[int, ...] = (2, 4, 8, 16, 32),
    aperture_radius: float = 150.0,
    projected_weight: bool = True,
) -> list[RayTraceResult]:
    """按给定细分级别计算接收比收敛序列。"""

    if not levels or any(level < 1 for level in levels):
        raise ValueError("收敛级别必须全部为正")
    return [
        trace_surface(
            data,
            frame,
            nodes,
            subdivision=level,
            aperture_radius=aperture_radius,
            projected_weight=projected_weight,
        )
        for level in levels
    ]

```

```

def panel_centroid_landings(
    data: FastData,
    frame: DirectionFrame,
    nodes: FloatArray,
) -> FloatArray:
    """返回面板中心反射线在馈源平面内的二维正交坐标。"""

    vertices, _, reflected = _face_geometry(data, frame, nodes)
    centroids = np.mean(vertices, axis=1)
    denominator = reflected @ frame.sky
    numerator = (frame.focus[None, :] - centroids) @ frame.sky
    distance = np.divide(
        numerator,
        denominator,
        out=np.full_like(numerator, np.nan),
        where=np.abs(denominator) > 1e-14,
    )
    intersections = centroids + distance[:, None] * reflected
    reference = np.array([1.0, 0.0, 0.0])
    if abs(float(reference @ frame.sky)) > 0.9:
        reference = np.array([0.0, 1.0, 0.0])
    first = np.cross(frame.sky, reference)
    first /= np.linalg.norm(first)
    second = np.cross(frame.sky, first)
    offsets = intersections - frame.focus[None, :]
    return np.column_stack((offsets @ first, offsets @ second))

```

正式实验与绘图入口

g3_run.py

```

"""G3 正式实验入口。

入口读取真实附件，完成理想面比较、约束优化、接收比与稳健性分析，
保存可追溯结果表，并仅从落盘数据生成论文用矢量图。
"""

from __future__ import annotations

import argparse
import importlib.metadata
import json
import sys
import time
from pathlib import Path
from typing import Any

import numpy as np
import pandas as pd

from fast_geometry import (
    DirectionFrame,
    FastData,
    build_direction_frame,
    load_fast_data,
)
from g3_plot import (
    plot_constraint_usage,
    plot_displacement_and_stroke,
    plot_focal_search,
    plot_receiver_results,
    plot_sensitivity,

```

```

)
from optimization import OptimizationResult, optimize_work_surface
from problem1 import FocalModelComparison, compare_focal_models
from raytrace import panel_centroid_landings, trace_convergence, trace_surface

def _parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="运行 FAST G3 正式实验")
    parser.add_argument("--data-dir", type=Path, required=True)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--figures-dir", type=Path, required=True)
    return parser

def _focal_payload(
    data: FastData,
    frame: DirectionFrame,
    comparison: FocalModelComparison,
) -> dict[str, Any]:
    selected = comparison.hardware_result
    focus_radius = float(np.linalg.norm(frame.focus))
    return {
        "geometric_focal_length_m": comparison.geometric_focal_length,
        "least_squares_focal_length_m": comparison.least_squares_focal_length,
        "selected_focal_length_m": selected.focal_length,
        "selected_vertex_radius_m": focus_radius + selected.focal_length,
        "selected_vertex_xyz_m": (
            (focus_radius + selected.focal_length) * frame.axis
        ).tolist(),
        "candidate_metrics": comparison.metrics,
        "grid_focal_lengths_m": selected.grid_focal_lengths.tolist(),
        "grid_hardware_utilization": selected.grid_utilizations.tolist(),
        "active_nodes": int(np.sum(frame.active_nodes)),
        "sphere_radius_m": data.radius,
    }

def _save_solution_tables(
    data: FastData,
    frame: DirectionFrame,
    target: np.ndarray,
    solution: OptimizationResult,
    output_dir: Path,
) -> dict[str, Path]:
    active = frame.active_nodes
    adjusted = solution.adjusted_nodes[active]
    axial = data.nodes @ frame.axis
    rho = np.sqrt(np.maximum(0.0, np.sum(data.nodes**2, axis=1) - axial**2))
    nodes_frame = pd.DataFrame(
        {
            "node_id": data.node_ids[active],
            "x_m": adjusted[:, 0],
            "y_m": adjusted[:, 1],
            "z_m": adjusted[:, 2],
            "rho_m": rho[active],
            "displacement_m": solution.displacement[active],
            "target_displacement_m": target[active],
            "stroke_m": solution.strokes[active],
        }
    )
    nodes_path = output_dir / "problem2_nodes.csv"
    nodes_frame.to_csv(nodes_path, index=False, encoding="utf-8")

```

```

edges = data.edges[frame.incident_edges]
baseline_lengths = data.edge_lengths[frame.incident_edges]
differences = solution.adjusted_nodes[edges[:, 0]] -
solution.adjusted_nodes[edges[:, 1]]
adjusted_lengths = np.linalg.norm(differences, axis=1)
relative = adjusted_lengths / baseline_lengths - 1.0
edge_frame = pd.DataFrame(
    {
        "left_node_id": data.node_ids[edges[:, 0]],
        "right_node_id": data.node_ids[edges[:, 1]],
        "baseline_length_m": baseline_lengths,
        "adjusted_length_m": adjusted_lengths,
        "relative_change": relative,
        "absolute_relative_change": np.abs(relative),
    }
)
edges_path = output_dir / "constraint_edges.csv"
edge_frame.to_csv(edges_path, index=False, encoding="utf-8")
history_path = output_dir / "optimization_history.csv"
pd.DataFrame(solution.histories).to_csv(history_path, index=False,
encoding="utf-8")
return {"nodes": nodes_path, "edges": edges_path, "history": history_path}

def _save_workbook_payload(
    data: FastData,
    frame: DirectionFrame,
    solution: OptimizationResult,
    vertex_xyz: list[float],
    output_dir: Path,
) -> Path:
    """保存供 artifact-tool 写入竞赛模板的纯 JSON 数据。"""

    active = frame.active_nodes
    adjusted = solution.adjusted_nodes[active]
    payload = {
        "vertex_xyz_m": vertex_xyz,
        "adjusted_nodes": [
            [str(node_id), float(x), float(y), float(z)]
            for node_id, (x, y, z) in zip(
                data.node_ids[active],
                adjusted,
                strict=True,
            )
        ],
        "actuator_strokes": [
            [str(node_id), float(stroke)]
            for node_id, stroke in zip(
                data.node_ids[active],
                solution.strokes[active],
                strict=True,
            )
        ],
    }
    payload_path = output_dir / "workbook_payload.json"
    payload_path.write_text(
        json.dumps(payload, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    return payload_path

def _trace_convergence_table(

```

```

data: FastData,
frame: DirectionFrame,
adjusted_nodes: np.ndarray,
output_dir: Path,
) -> tuple[dict[str, list[Any]], Path]:
    """保存基准面和工作面的射线细分收敛表。"""

    # 64 阶用于确认 0.5 米馈源半径下离散命中边界的收敛稳定性。
    levels = (2, 4, 8, 16, 32, 64)
    records: list[dict[str, Any]] = []
    series: dict[str, list[Any]] = {}
    for name, nodes in (("baseline", data.nodes), ("work", adjusted_nodes)):
        results = trace_convergence(data, frame, nodes, levels=levels)
        series[name] = results
        records.extend(
            {
                "surface": name,
                "subdivision": item.subdivision,
                "reception_ratio": item.reception_ratio,
                "received_weight": item.received_weight,
                "total_weight": item.total_weight,
                "included_samples": item.included_samples,
            }
            for item in results
        )
    convergence_path = output_dir / "raytrace_convergence.csv"
    pd.DataFrame(records).to_csv(convergence_path, index=False, encoding="utf-8")
    return series, convergence_path

def _trace_sensitivity_table(
    data: FastData,
    frame: DirectionFrame,
    adjusted_nodes: np.ndarray,
    output_dir: Path,
) -> Path:
    """保存口径半径与能量权重口径的敏感性结果。"""

    sensitivity_records: list[dict[str, Any]] = []
    for radius in (149.5, 150.0, 150.5):
        for name, nodes in (("baseline", data.nodes), ("work", adjusted_nodes)):
            for weighting, projected in (("projected", True), ("equal_panel", False)):
                result = trace_surface(
                    data,
                    frame,
                    nodes,
                    subdivision=16,
                    aperture_radius=radius,
                    projected_weight=projected,
                )
                sensitivity_records.append(
                    {
                        "surface": name,
                        "aperture_radius_m": radius,
                        "weighting": weighting,
                        "reception_ratio": result.reception_ratio,
                    }
                )
    sensitivity_path = output_dir / "raytrace_sensitivity.csv"
    pd.DataFrame(sensitivity_records).to_csv(
        sensitivity_path,
        index=False,
        encoding="utf-8",
    )

```

```

    )
    return sensitivity_path

def _trace_landing_table(
    data: FastData,
    frame: DirectionFrame,
    adjusted_nodes: np.ndarray,
    output_dir: Path,
) -> Path:
    """保存基准面和工作面的面板中心馈源平面落点。"""

    landing_records: list[pd.DataFrame] = []
    for name, nodes in (("baseline", data.nodes), ("work", adjusted_nodes)):
        landings = panel_centroid_landings(data, frame, nodes)
        landing_records.append(
            pd.DataFrame(
                {
                    "surface": name,
                    "receiver_x_m": landings[:, 0],
                    "receiver_y_m": landings[:, 1],
                }
            )
        )
    landings_path = output_dir / "receiver_landings.csv"
    pd.concat(landing_records, ignore_index=True).to_csv(
        landings_path,
        index=False,
        encoding="utf-8",
    )
    return landings_path

def _summarize_ray_series(series: dict[str, list[Any]]) -> dict[str, Any]:
    """从最高两档细分计算接收比及末步离散误差。"""

    levels = [item.subdivision for item in series["baseline"]]
    baseline_final = series["baseline"][-1].reception_ratio
    work_final = series["work"][-1].reception_ratio
    return {
        "subdivision_levels": levels,
        "baseline_reception_ratio": baseline_final,
        "work_reception_ratio": work_final,
        "absolute_improvement": work_final - baseline_final,
        "relative_improvement": (
            (work_final - baseline_final) / baseline_final
            if baseline_final > 0
            else None
        ),
        "baseline_last_change": abs(
            series["baseline"][-1].reception_ratio
            - series["baseline"][-2].reception_ratio
        ),
        "work_last_change": abs(
            series["work"][-1].reception_ratio
            - series["work"][-2].reception_ratio
        ),
    }

def _run_ray_experiments(
    data: FastData,
    frame: DirectionFrame,

```

```

    adjusted_nodes: np.ndarray,
    output_dir: Path,
) -> tuple[dict[str, Any], dict[str, Path]]:
    """运行收敛、敏感性和落点三组确定性射线实验。"""

    series, convergence_path = _trace_convergence_table(
        data,
        frame,
        adjusted_nodes,
        output_dir,
    )
    sensitivity_path = _trace_sensitivity_table(
        data,
        frame,
        adjusted_nodes,
        output_dir,
    )
    landings_path = _trace_landing_table(
        data,
        frame,
        adjusted_nodes,
        output_dir,
    )
    paths = {
        "convergence": convergence_path,
        "sensitivity": sensitivity_path,
        "landings": landings_path,
    }
    return _summarize_ray_series(series), paths

def _make_figures(
    summary_path: Path,
    tables: dict[str, Path],
    ray_paths: dict[str, Path],
    figures_dir: Path,
) -> list[Path]:
    figures = [
        figures_dir / "focal_model_comparison.pdf",
        figures_dir / "displacement_and_stroke.pdf",
        figures_dir / "constraint_usage.pdf",
        figures_dir / "receiver_results.pdf",
        figures_dir / "raytrace_sensitivity.pdf",
    ]
    plot_focal_search(summary_path, figures[0])
    plot_displacement_and_stroke(tables["nodes"], figures[1])
    plot_constraint_usage(tables["edges"], figures[2])
    plot_receiver_results(ray_paths["convergence"], ray_paths["landings"], figures[3])
    plot_sensitivity(ray_paths["sensitivity"], figures[4])
    return figures

def _environment_payload() -> dict[str, Any]:
    """记录正式实验所用解释器与关键科学计算包版本。"""

    package_names = (
        "numpy",
        "scipy",
        "pandas",
        "matplotlib",
        "cvxpy",
        "osqp",
        "clarabel",

```

```

    )
    return {
        "python_executable": sys.executable,
        "python_version": sys.version,
        "packages": {
            name: importlib.metadata.version(name)
            for name in package_names
        },
    }
}

def _optimization_payload(
    solution: OptimizationResult,
    frame: DirectionFrame,
) -> dict[str, Any]:
    """提取优化误差、约束占用率和促动器范围。"""

    active = frame.active_nodes
    return {
        "objective_mse_m2": solution.objective_mse_m2,
        "rmse_m": float(np.sqrt(solution.objective_mse_m2)),
        "max_abs_error_m": solution.max_abs_error_m,
        "selected_start": solution.selected_start,
        "constraint": solution.constraint,
        "displacement_min_m": float(np.min(solution.displacement[active])),
        "displacement_max_m": float(np.max(solution.displacement[active])),
        "stroke_min_m": float(np.min(solution.strokes[active])),
        "stroke_max_m": float(np.max(solution.strokes[active])),
        "solver_statuses": sorted(set(solution.solver_statuses)),
    }

def run_formal_experiment(
    data_dir: Path,
    output_dir: Path,
    figures_dir: Path,
) -> dict[str, Any]:
    """运行 G3 正式模型、保存结果数据并生成矢量图。"""

    started = time.perf_counter()
    output_dir.mkdir(parents=True, exist_ok=True)
    figures_dir.mkdir(parents=True, exist_ok=True)
    data = load_fast_data(data_dir)
    frame1 = build_direction_frame(data, 0.0, 90.0)
    frame2 = build_direction_frame(data, 36.795, 78.169)
    comparison1 = compare_focal_models(data, frame1)
    comparison2 = compare_focal_models(data, frame2)
    target2 = comparison2.hardware_result.displacement
    solution = optimize_work_surface(data, frame2, target2)
    tables = _save_solution_tables(data, frame2, target2, solution, output_dir)
    vertex_xyz = _focal_payload(data, frame2, comparison2)["selected_vertex_xyz_m"]
    workbook_payload = _save_workbook_payload(
        data,
        frame2,
        solution,
        vertex_xyz,
        output_dir,
    )
    ray_summary, ray_paths = _run_ray_experiments(
        data,
        frame2,
        solution.adjusted_nodes,
        output_dir,
    )

```

```

)
summary: dict[str, Any] = {
    "stage": "g3-formal",
    "status": "FORMAL_RESULTS",
    "environment": _environment_payload(),
    "problem1": {"focal_models": _focal_payload(data, frame1, comparison1)},
    "problem2": {
        "focal_models": _focal_payload(data, frame2, comparison2),
        "optimization": _optimization_payload(solution, frame2),
    },
    "problem3": ray_summary,
    "runtime_seconds": None,
}
summary_path = output_dir / "formal_results.json"
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding="utf-8")
figures = _make_figures(summary_path, tables, ray_paths, figures_dir)
summary["runtime_seconds"] = time.perf_counter() - started
summary["artifacts"] = {
    "tables": {key: str(value) for key, value in tables.items()},
    "ray_tables": {key: str(value) for key, value in ray_paths.items()},
    "figures": [str(value) for value in figures],
    "workbook_payload": str(workbook_payload),
}
summary_path.write_text(json.dumps(summary, ensure_ascii=False, indent=2),
encoding="utf-8")
return summary

def main() -> None:
    """解析命令行并运行 G3 正式实验。"""

    arguments = _parser().parse_args()
    result = run_formal_experiment(
        arguments.data_dir,
        arguments.output_dir,
        arguments.figures_dir,
    )
    print(
        json.dumps(
            {
                "status": result["status"],
                "work_reception_ratio": result["problem3"]["work_reception_ratio"],
                "baseline_reception_ratio": result["problem3"]
["baseline_reception_ratio"],
                "runtime_seconds": result["runtime_seconds"],
            },
            ensure_ascii=False,
        )
    )

if __name__ == "__main__":
    main()

```

g3_plot.py

```

"""从 G3 已保存结果生成论文用数据图。

本模块不重新运行模型，只读取 JSON/CSV 结果并输出矢量 PDF。
"""

from __future__ import annotations

```

```

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def _configure_chinese_font() -> None:
    # 图表统一使用可嵌入的中文字体，并保留坐标轴负号的数学含义。
    plt.rcParams["font.sans-serif"] = [
        "Microsoft YaHei",
        "SimHei",
        "Noto Sans CJK SC",
        "Arial Unicode MS",
    ]
    plt.rcParams["axes.unicode_minus"] = False

def plot_focal_search(summary_path: Path, output_path: Path) -> None:
    """绘制两个方向的焦距-硬件占用率曲线。"""

    summary = json.loads(summary_path.read_text(encoding="utf-8"))
    _configure_chinese_font()
    figure, axes = plt.subplots(1, 2, figsize=(9.0, 3.4), constrained_layout=True)
    for axis, key, label in zip(
        axes,
        ("problem1", "problem2"),
        ("正上方观测", "指定方向观测"),
        strict=True,
    ):
        values = summary[key]["focal_models"]
        axis.plot(
            values["grid_focal_lengths_m"],
            values["grid_hardware_utilization"],
            color="#0072B2",
            linewidth=1.6,
            label="最大硬件占用率",
        )
        axis.axvline(
            values["selected_focal_length_m"],
            color="#D55E00",
            linestyle="--",
            linewidth=1.2,
            label="硬件友好焦距",
        )
        axis.axvline(
            values["least_squares_focal_length_m"],
            color="#009E73",
            linestyle="-.",
            linewidth=1.1,
            label="最小二乘焦距",
        )
        axis.axhline(1.0, color="#444444", linestyle=":", linewidth=1.0)
        axis.set_xlabel("抛物面焦距 (米)")
        axis.set_ylabel("最大归一化硬件占用率")
        axis.text(0.03, 0.94, label, transform=axis.transAxes, va="top")
        axis.grid(alpha=0.22)
    axes[0].legend(fontsize=8)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    figure.savefig(output_path, format="pdf")

```

```

plt.close(figure)

def plot_displacement_and_stroke(nodes_path: Path, output_path: Path) -> None:
    """绘制节点目标/实际位移与促动器伸缩量分布。"""

    nodes = pd.read_csv(nodes_path)
    _configure_chinese_font()
    figure, axes = plt.subplots(1, 2, figsize=(9.0, 3.5), constrained_layout=True)
    order = np.argsort(nodes["rho_m"].to_numpy())
    axes[0].plot(
        nodes["rho_m"].to_numpy()[order],
        nodes["target_displacement_m"].to_numpy()[order],
        color="#999999",
        linewidth=1.0,
        label="理想目标",
    )
    axes[0].scatter(
        nodes["rho_m"],
        nodes["displacement_m"],
        s=8,
        color="#0072B2",
        alpha=0.65,
        label="约束优化后",
    )
    axes[0].set_xlabel("离观测轴投影距离 (米)")
    axes[0].set_ylabel("向球心径向位移 (米)")
    axes[0].legend(fontsize=8)
    axes[0].grid(alpha=0.22)
    axes[1].hist(
        nodes["stroke_m"],
        bins=32,
        color="#009E73",
        edgecolor="white",
        linewidth=0.4,
    )
    axes[1].axvline(-0.6, color="#D55E00", linestyle="--", linewidth=1.0)
    axes[1].axvline(0.6, color="#D55E00", linestyle="--", linewidth=1.0)
    axes[1].set_xlabel("促动器向球心伸缩量 (米)")
    axes[1].set_ylabel("节点数")
    axes[1].grid(axis="y", alpha=0.22)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    figure.savefig(output_path, format="pdf")
    plt.close(figure)

def plot_constraint_usage(edges_path: Path, output_path: Path) -> None:
    """绘制主索相对变化的排序曲线和分布。"""

    edges = pd.read_csv(edges_path)
    values = np.sort(edges["absolute_relative_change"].to_numpy())
    _configure_chinese_font()
    figure, axes = plt.subplots(1, 2, figsize=(9.0, 3.4), constrained_layout=True)
    axes[0].plot(np.arange(1, len(values) + 1), 100.0 * values, color="#0072B2")
    axes[0].axhline(0.07, color="#D55E00", linestyle="--", label="0.07% 上限")
    axes[0].set_xlabel("相关主索 (按相对变化排序)")
    axes[0].set_ylabel("绝对相对长度变化 (%)")
    axes[0].legend(fontsize=8)
    axes[0].grid(alpha=0.22)
    axes[1].hist(100.0 * values, bins=36, color="#56B4E9", edgecolor="white")
    axes[1].axvline(0.07, color="#D55E00", linestyle="--")
    axes[1].set_xlabel("绝对相对长度变化 (%)")

```

```

axes[1].set_ylabel("主索数")
axes[1].grid(axis="y", alpha=0.22)
output_path.parent.mkdir(parents=True, exist_ok=True)
figure.savefig(output_path, format="pdf")
plt.close(figure)

def plot_receiver_results(
    convergence_path: Path,
    landings_path: Path,
    output_path: Path,
) -> None:
    """绘制接收比收敛和面板中心落点。"""

    convergence = pd.read_csv(convergence_path)
    landings = pd.read_csv(landings_path)
    _configure_chinese_font()
    figure, axes = plt.subplots(1, 2, figsize=(9.0, 3.7), constrained_layout=True)
    for surface, color, label in (
        ("baseline", "#999999", "基准球面"),
        ("work", "#0072B2", "调节工作面"),
    ):
        subset = convergence[convergence["surface"] == surface]
        axes[0].plot(
            subset["subdivision"],
            subset["reception_ratio"],
            marker="o",
            color=color,
            label=label,
        )
        axes[0].set_xscale("log", base=2)
        axes[0].set_xlabel("三角边细分级别")
        axes[0].set_ylabel("接收比")
        axes[0].legend(fontsize=8)
        axes[0].grid(alpha=0.22)
        for surface, color, label in (
            ("baseline", "#999999", "基准球面"),
            ("work", "#0072B2", "调节工作面"),
        ):
            subset = landings[landings["surface"] == surface]
            axes[1].scatter(
                subset["receiver_x_m"],
                subset["receiver_y_m"],
                s=5,
                alpha=0.35,
                color=color,
                label=label,
            )
        circle = plt.Circle((0, 0), 0.5, fill=False, color="#D55E00", linewidth=1.2)
        axes[1].add_patch(circle)
        axes[1].set_aspect("equal", adjustable="box")
        axes[1].set_xlim(-3.0, 3.0)
        axes[1].set_ylim(-3.0, 3.0)
        axes[1].set_xlabel("馈源平面横向坐标 (米)")
        axes[1].set_ylabel("馈源平面纵向坐标 (米)")
        axes[1].legend(fontsize=8)
        axes[1].grid(alpha=0.22)
        output_path.parent.mkdir(parents=True, exist_ok=True)
        figure.savefig(output_path, format="pdf")
        plt.close(figure)

```

```
def plot_sensitivity(sensitivity_path: Path, output_path: Path) -> None:
    """绘制口径边界和权重口径的接收比敏感性。"""

    sensitivity = pd.read_csv(sensitivity_path)
    _configure_chinese_font()
    figure, axis = plt.subplots(figsize=(5.4, 3.5), constrained_layout=True)
    for surface, color, label in (
        ("baseline", "#999999", "基准球面"),
        ("work", "#0072B2", "调节工作面"),
    ):
        subset = sensitivity[
            (sensitivity["surface"] == surface)
            & (sensitivity["weighting"] == "projected")
        ]
        axis.plot(
            subset["aperture_radius_m"],
            subset["reception_ratio"],
            marker="o",
            color=color,
            label=label,
        )
    axis.set_xlabel("投影口径半径 (米)")
    axis.set_ylabel("接收比")
    axis.legend(fontsize=8)
    axis.grid(alpha=0.22)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    figure.savefig(output_path, format="pdf")
    plt.close(figure)
```

最小验证入口

g2_smoke.py

```
"""G2 最小可行入口。

该入口读取真实附件，完成两个方向的理想抛物面一维搜索、问题二
可行基线选择、解析聚焦测试，并保存 JSON 与诊断 PDF。
"""

from __future__ import annotations

import argparse
import importlib.metadata
import json
import sys
import time
from pathlib import Path
from typing import Any

import numpy as np

from fast_geometry import build_direction_frame, load_fast_data
from g2_plot import plot_g2_diagnostic
from problem1 import FocalSearchResult, search_hardware_friendly_focal_length
from problem2 import choose_feasible_baseline
from problem3 import ideal_paraboloid_focus_error

def _parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="运行 FAST G2 最小可行验证")
    parser.add_argument("--data-dir", type=Path, required=True)
    parser.add_argument("--output", type=Path, required=True)
```

```

parser.add_argument("--figure", type=Path, required=True)
return parser

def _search_payload(search: FocalSearchResult) -> dict[str, Any]:
    return {
        "focal_length_m": search.focal_length,
        "vertex_radius_m": None,
        "optimizer_success": search.optimizer_success,
        "optimizer_message": search.optimizer_message,
        "constraint": search.constraint,
        "target_displacement_min_m": float(np.min(search.displacement)),
        "target_displacement_max_m": float(np.max(search.displacement)),
        "grid_focal_lengths_m": search.grid_focal_lengths.tolist(),
        "grid_hardware_utilization": search.grid_utilizations.tolist(),
    }

def run_g2_smoke(data_dir: Path, output: Path, figure: Path) -> dict[str, Any]:
    """执行 G2 最小验证并返回与保存的结果字典。

    参数:
        data_dir: 项目内原始附件目录。
        output: G2 JSON 结果路径。
        figure: G2 诊断 PDF 路径。

    返回:
        已写入 JSON 的可审计结果。
    """

    started = time.perf_counter()
    data = load_fast_data(data_dir)
    # 两个问题复用同一模型定义, 仅观测轴和离散工作口径不同。
    frame1 = build_direction_frame(data, 0.0, 90.0)
    frame2 = build_direction_frame(data, 36.795, 78.169)
    search1 = search_hardware_friendly_focal_length(data, frame1)
    search2 = search_hardware_friendly_focal_length(data, frame2)
    baseline2 = choose_feasible_baseline(data, frame2, search2.displacement)
    focus_error1 = ideal_paraboloid_focus_error(frame1, search1.focal_length)
    focus_error2 = ideal_paraboloid_focus_error(frame2, search2.focal_length)
    payload1 = _search_payload(search1)
    payload2 = _search_payload(search2)
    focus_radius = float(np.linalg.norm(frame1.focus))
    payload1["vertex_radius_m"] = focus_radius + search1.focal_length
    payload2["vertex_radius_m"] = focus_radius + search2.focal_length
    result: dict[str, Any] = {
        "stage": "g2-minimal-validation",
        "status": "BASELINE_ONLY",
        "environment": {
            "python_executable": sys.executable,
            "python_version": sys.version,
            "packages": {
                name: importlib.metadata.version(name)
                for name in ("numpy", "scipy", "pandas", "matplotlib")
            },
        },
        "data": {
            "nodes": len(data.node_ids),
            "panels": len(data.panels),
            "edges": len(data.edges),
            "radius_m": data.radius,
        },
    },

```

```

        "problem1": {
            **payload1,
            "active_nodes": int(np.sum(frame1.active_nodes)),
            "focus_error_m": focus_error1,
        },
        "problem2": {
            **payload2,
            "active_nodes": int(np.sum(frame2.active_nodes)),
            "focus_error_m": focus_error2,
            "baseline_source": baseline2.source,
            "baseline_constraint": baseline2.constraint,
            "baseline_mean_square_error_m2": baseline2.weighted_mean_square_error,
            "baseline_max_abs_target_error_m": baseline2.max_abs_target_error_m,
        },
        "runtime_seconds": time.perf_counter() - started,
        "limitations": [
            "G2 只使用固定半宽的一维搜索，不是 G3 自动扩张正式搜索。",
            "G2 仅选择目标面或零位移可行基线，未运行正式顺序凸化。",
            "G2 只验证解析聚焦公式，未计算正式面板接收比。",
        ],
    }
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(json.dumps(result, ensure_ascii=False, indent=2),
encoding="utf-8")
    plot_g2_diagnostic(output, figure)
    return result

def main() -> None:
    """解析命令行并运行 G2 最小验证。"""

    arguments = _parser().parse_args()
    result = run_g2_smoke(arguments.data_dir, arguments.output, arguments.figure)
    print(
        json.dumps(
            {
                "status": result["status"],
                "problem1_feasible": result["problem1"]["constraint"]["feasible"],
                "problem2_baseline": result["problem2"]["baseline_source"],
                "runtime_seconds": result["runtime_seconds"],
            },
            ensure_ascii=False,
        )
    )

if __name__ == "__main__":
    main()

```

g2_plot.py

```

"""从已保存的 G2 JSON 结果生成最小诊断图。"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt

def plot_g2_diagnostic(result_path: Path, output_path: Path) -> None:

```

```

"""绘制两个观测方向的一维硬件占用率搜索曲线。"""

# 绘图只消费已落盘 JSON，避免作图阶段重新计算或改写核心指标。
result = json.loads(result_path.read_text(encoding="utf-8"))
plt.rcParams["font.sans-serif"] = [
    "Microsoft YaHei",
    "SimHei",
    "Noto Sans CJK SC",
    "Arial Unicode MS",
]
plt.rcParams["axes.unicode_minus"] = False
figure, axes = plt.subplots(1, 2, figsize=(9.0, 3.4), constrained_layout=True)
for axis, key, label in zip(
    axes,
    ("problem1", "problem2"),
    ("问题一：正上方", "问题二：指定方向"),
    strict=True,
):
    values = result[key]
    focal = values["grid_focal_lengths_m"]
    utilization = values["grid_hardware_utilization"]
    axis.plot(focal, utilization, color="#0072B2", linewidth=1.6)
    axis.axvline(
        values["focal_length_m"],
        color="#D55E00",
        linestyle="--",
        linewidth=1.2,
        label="G2 最优焦距",
    )
    axis.axhline(1.0, color="#444444", linestyle=":", linewidth=1.0, label="硬约束上限")

    axis.set_xlabel("抛物面焦距 (米)")
    axis.set_ylabel("最大归一化硬件占用率")
    axis.set_title(label, fontsize=10)
    axis.grid(alpha=0.25)
axes[0].legend(fontsize=8)
output_path.parent.mkdir(parents=True, exist_ok=True)
figure.savefig(output_path, format="pdf")
plt.close(figure)

```